



Universidad
Señor de Sipán

Taller de Lenguajes de Programación

Sesión 01:

Introducción a Python y Backend



Introducción

El propósito de esta primera sesión es que te familiarizes con lo básico de Python para luego llevarte paso a paso en el desarrollo de aplicaciones web con Django y Django Rest Framework (DRF), para ello es importante que realices los ejercicios que te planteo en esta guía.

En esta sesión introductoria vamos a ver los siguientes temas:

1. Funciones en Python
2. Statements y Funciones de Control
3. Tipos de datos complejos: Listas, diccionarios y tuplas
4. Decoradores
5. Control de Excepciones
6. Programación Orientada a objetos

Adicionalmente, para esta sesión vamos a ejecutar nuestros ejercicios en cualquiera de las siguientes alternativas

1. Editor online: <https://onecompiler.com/python/>
2. Visual Studio Code de manera local

Funciones en Python

Una función es un bloque de código que realiza una tarea específica. Supongamos que necesitamos crear un programa para crear un círculo y colorear. Podemos crear dos funciones para resolver este problema:

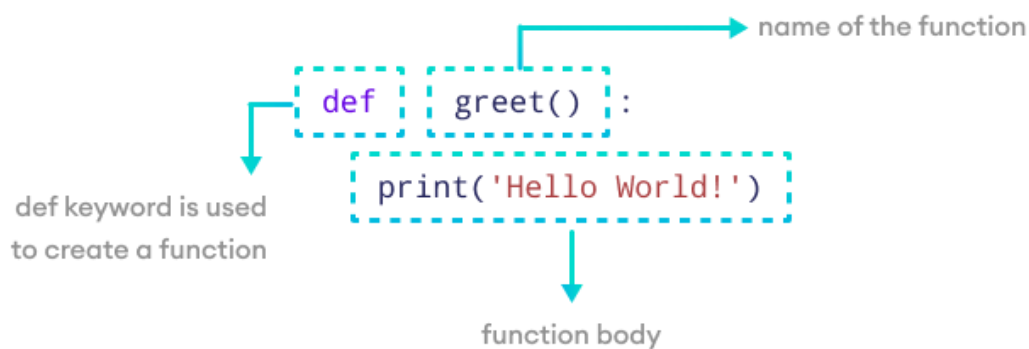
- función para crear un círculo
- función para colorear la forma

Dividir un problema complejo en porciones más pequeñas hace que nuestro programa sea fácil de entender y reutilizar. **Una función es un bloque de código reutilizable que realiza una tarea específica. Nos ayuda a evitar la repetición, mejorar la organización del código y facilitar el mantenimiento.**

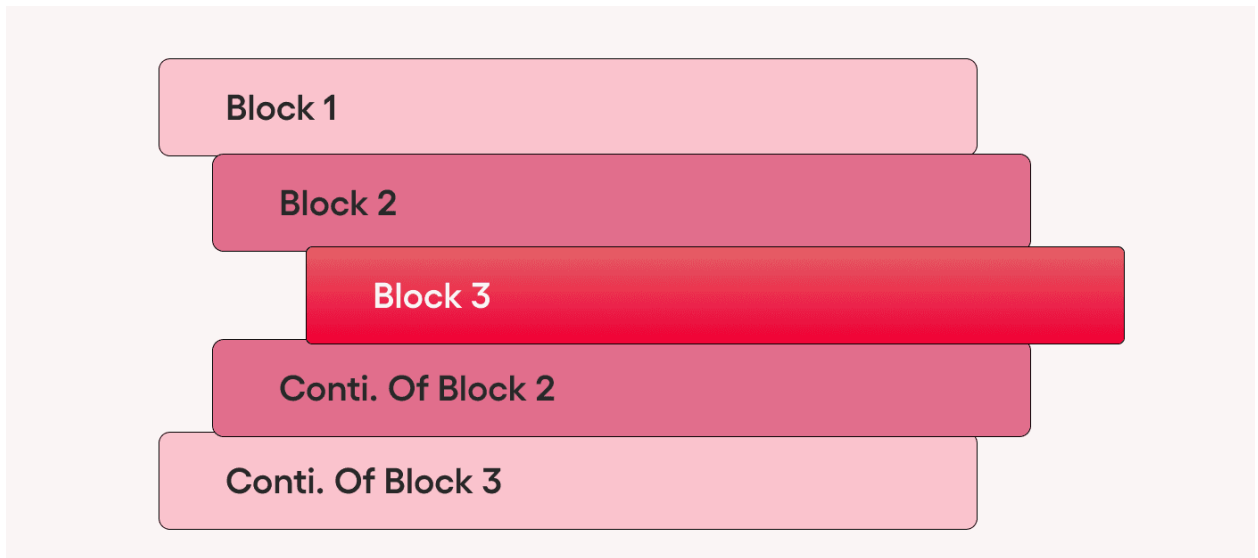
Creando una función

Ahora vamos a crear una función greet

```
def greet():  
    print('Hello World!')
```



Nota: Cuando escribas una función, presta atención a la sangría, que son los espacios al principio de una línea de código.



Llamando una función

Si ejecutamos el código anterior, no obtendremos ninguna salida. Es porque crear una función no significa que estamos ejecutando el código dentro de ella. Significa que el código está ahí para que lo usemos si queremos.

Para utilizar esta función, tenemos que llamar a la función.

Function Call

```
greet()
```

Ejemplo de la llamada a la función

main.py



```
1 def greet():  
2     print ('hello world!!!')  
3  
4     print('Outside the funcion')  
5  
6     greet()  
7
```

Output:

Output:

```
Outside the funcion  
hello world!!!
```

Argumentos y parámetros de una función

Los argumentos son entradas que se dan a la función para sean procesados dentro de la función

main.py



```
1  
2 def greet(name):  
3     print('hello ', name)  
4  
5     #pass arguments  
6     greet('Javier')  
7
```

Cómo resultado de la ejecución de este código debemos obtener el siguiente resultado

Output:

```
hello  Javier
```

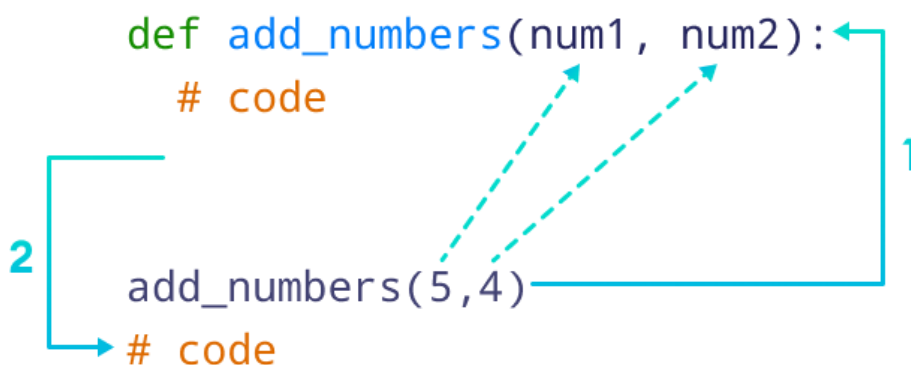
Aquí, pasamos “Javier” como argumento a la función greet().

Podemos pasar diferentes argumentos en cada llamada, haciendo la función reutilizable y dinámica.

Ejemplo de función: Sumar dos números

```
def add_numbers(num1, num2):  
    sum = num1 + num2  
    return sum
```

```
#call to function  
sum = add_numbers(5,4)  
print('Total', sum)
```



5 tipos de argumentos de funciones de Python

1. Argumentos por defecto

2. Argumentos de palabra clave
3. Argumentos posicionales
4. Argumentos posicionales arbitrarios
5. Argumentos de palabra clave arbitrarios

1. Argumentos por defecto

Cuando se escribe una función, se puede definir un valor por defecto para cada parámetro. Si se proporciona un argumento para un parámetro en la llamada a la función, Python utiliza el valor del argumento. Si no, utiliza el valor por defecto del parámetro. Así que cuando defines un valor por defecto para un parámetro, puedes excluir el argumento correspondiente que normalmente escribirías en la llamada a la función. El uso de valores por defecto puede simplificar las llamadas a funciones y aclarar la forma en que éstas se utilizan normalmente.

Por ejemplo creamos la siguiente función

```
Python

def sumatoria(num1, num2 = 3, num3 = 5):

    return num1+num2+num3
```

En la función observamos tres parámetros o argumentos. Nótese que los argumentos por defecto son aquellos que ya tienen un valor predeterminado por lo cual en caso de no enviarse estos parámetros al momento de la llamada, dentro de la función tomará los valores especificados por “defecto”. Así por ejemplo hacemos la llamada a nuestra función con solo un parámetro

```
#call to function
print(sumatoria(5))
```

Obtenemos el siguiente resultado:

Output:

13

Ahora invocamos nuevamente a la función enviando ahora si a los tres parámetros:

```
#call to function  
print(sumatoria(5,10,8))
```

Output:

23

2. Argumentos keyword

Un argumento de palabra clave es un par nombre-valor que se pasa a una función. Se asocian directamente el nombre y el valor dentro del argumento, de modo que cuando se pasa el argumento a la función, no hay confusión. Los argumentos con palabras clave te liberan de tener que preocuparte de ordenar correctamente tus argumentos en la llamada a la función, y aclaran el papel de cada valor en la llamada a la función.

Veamos con un ejemplo para ello crearemos una función

Python

```
def describe_pet(animal_type, pet_name):  
    """Display information about a pet."""  
    print(f"\nI have a {animal_type}.")  
    print(f"My {animal_type}'s name is {pet_name.title()}")
```

Ahora veamos el llamado de nuestra función

Python

```
describe_pet('Dog', 'Duque')
```

Obtendremos el siguiente resultado

```
I have a Dog.  
My Dog's name is Duque.
```

Ahora probemos ejecutando nuestro código, pero cambiando el orden de los parámetros

Python

```
describe_pet(pet_name='Duque', animal_type='Cat')
```

Y obtenemos el siguiente resultado

```
I have a Cat.  
My Cat's name is Duque.
```

Nótese que en esta segunda invocación cambiamos el orden de los parámetros, pero eso no afectó la correcta ejecución de nuestra función, esto debido a que usamos argumentos con nombre clave al momento de la invocación por lo cual ya no es necesario respetar el orden con el que los parámetros fueron declarados en la función.

3. Argumentos posicionales

Cuando llamas a una función, Python debe hacer coincidir cada argumento de la llamada a la función con un parámetro de la definición de la función. La forma más sencilla de hacerlo es basándose en el orden de los argumentos proporcionados. Para ver cómo funciona esto, considere una función que muestra información sobre mascotas. La función dice

Python

```
def describe_pet(animal_type, pet_name):  
    """Display information about a pet."""  
    print(f"\nI have a {animal_type}.")
```

```
print(f"My {animal_type}'s name is {pet_name.title()}.")
```

Ahora veamos la ejecución o la llamada a nuestra función

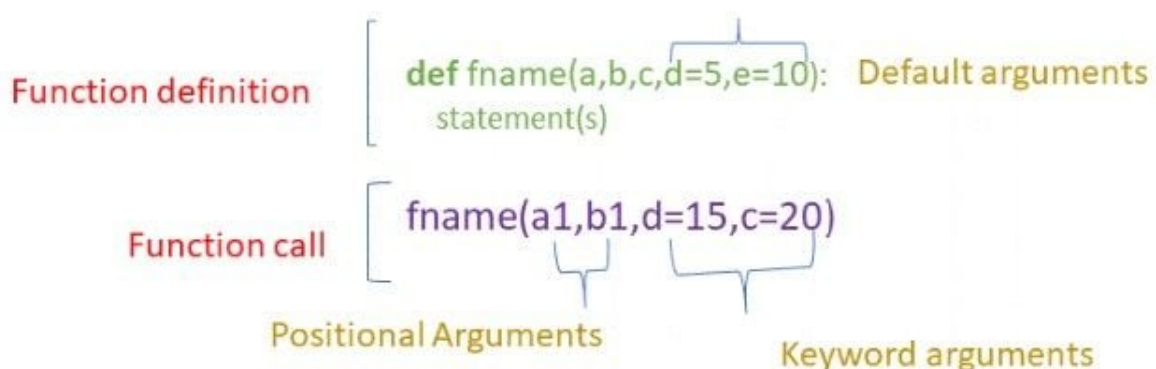
```
Python  
describe_pet('Dog', 'Duque')
```

El resultado sería

```
I have a Dog.  
My Dog's name is Duque.
```

La definición muestra que esta función necesita un tipo de animal y el nombre del animal. Cuando llamamos a `describe_pet()`, necesitamos proporcionar un tipo de animal y un nombre, en ese orden. Por ejemplo, en la llamada a la función, el argumento «Dog» se asigna al parámetro `pet_type` y el argumento «Duque» se asigna al parámetro `pet_name`. En el cuerpo de la función, estos dos parámetros se utilizan para mostrar información sobre la mascota que se está describiendo.

Default vs. Keyword vs. Positional Arguments



Puntos importantes a tener en cuenta para los argumentos por defecto, por palabra clave y posicionales

- 
1. Los argumentos por defecto deben seguir a los que no lo son
 2. Los argumentos de palabras clave deben seguir a los argumentos posicionales
 3. Todos los argumentos de palabra clave pasados deben coincidir con uno de los argumentos aceptados por la función, y su orden no es importante.
 4. Ningún argumento debe recibir valor más de una vez
 5. Los argumentos por defecto son opcionales



```
1.default arguments should follow non-default arguments
2.keyword arguments should follow positional arguments
3.All the keyword arguments passed must match one of the
arguments accepted by the function and their order is not important.
4.No argument should receive a value more than once
5.Default arguments are optional arguments.
```

Los argumentos por defecto deben seguir a los que no lo son

Python

```
def add_numbers(a=2,b,c):
    return (a+b+c)

print('La suma es', add_numbers(3,4))
```

Output:

File "/main.py", line 1

```
def add_numbers(a=2,b,c):  
    ^
```

SyntaxError: parameter without a default follows parameter with a default

Los argumentos de palabras clave deben seguir a los argumentos posicionales

Python

```
def add_numbers(a,b,c):  
    return (a+b+c)  
  
print('La suma es', add_numbers(a=5,3,4))
```

Output:

File "/main.py", line 4

```
print('La suma es', add_numbers(a=5,3,4))  
    ^
```

SyntaxError: positional argument follows keyword argument

Todos los argumentos de palabra clave pasados deben coincidir con uno de los argumentos aceptados por la función, y su orden no es importante.

Python

```
def add_numbers(a,b,c):  
    return (a+b+c)  
  
print('La suma es', add_numbers(a=3,b1=4,c=5))
```

Output:

Traceback (most recent call last):

File "/main.py", line 4, in <module>

print('La suma es', add_numbers(a=3,b1=4,c=5))

^^

TypeError: add_numbers() got an unexpected keyword argument 'b1'

Ningún argumento debe recibir valor más de una vez

Python

```
def add_numbers(a,b,c):
```

```
    return (a+b+c)
```

```
print('La suma es', add_numbers(a=3,b1=4,c=5,a=2))
```

File "/main.py", line 4

```
print('La suma es', add_numbers(a=3,b1=4,c=5,a=2))
```

^^^

SyntaxError: keyword argument repeated: a

Los argumentos por defecto son opcionales

Python

```
def add_numbers(a,b,c=2):
```

```
    return (a+b+c)
```

```
print('La suma es', add_numbers(2,4))
```

6. Argumentos posicionales arbitrarios en Python

Los argumentos de longitud variable también se conocen como argumentos arbitrarios. Si no conocemos de antemano el número de argumentos necesarios para la función, podemos utilizar argumentos arbitrarios. Los argumentos arbitrarios son de dos tipos: argumentos de posición arbitraria y argumentos de palabra clave arbitraria.

En el caso de los argumentos arbitrarios posicionales, se coloca un asterisco (*) delante de un parámetro en la definición de la función que puede contener argumentos de longitud variable que no sean palabras clave. Estos argumentos se envolverán en una tupla. Antes del número variable de argumentos, puede haber cero o más argumentos normales.

```
Python

def add(*b):
    result=0
    for i in b:
        result=result+i
    return result

print (add(1,2,3,4,5))

#Output:15

print (add(10,20))

#Output:30
```

7. Argumentos arbitrarios de palabras clave en Python

Los argumentos arbitrarios de palabra clave (**kwargs) son argumentos arbitrarios con nombre. Sintaxis de la palabra clave argumento (keyword=argumento). La sintaxis **kwargs nos permite pasar un número diferente de argumentos con nombre.

Python

```
def fn(**a):  
    for i in a.items():  
        print (i)  
  
fn(numbers=5, colors="blue", fruits="apple")  
...
```

Output:

```
('numbers', 5)  
('colors', 'blue')  
('fruits', 'apple')  
...
```

Python

```
def some_function(**kwargs):  
    print(type(kwargs))  
    print(kwargs)  
  
some_function(first=11, second=22, some=33)
```

En el ejemplo anterior, pasamos los argumentos con nombre (first=11 y other) a la función some_function().

El kwargs tomado es un dict (diccionario) donde:

- claves son palabras clave tomadas en el tipo str.
- valores son valores de estas palabras clave.

Listas en Python

En Python, las listas nos permiten almacenar múltiples elementos en una sola variable. Por ejemplo, si necesitas almacenar las edades de todos los alumnos de una clase, puedes realizar esta tarea utilizando una lista.

Las listas son similares a las matrices (matrices dinámicas que nos permiten almacenar elementos de diferentes tipos de datos) en otros lenguajes de programación.

En esta sección aprenderás los diferentes métodos para crear una lista, añadir, modificar y eliminar elementos de la lista. Además, aprenderá cómo iterar la lista y acceder a los elementos de la lista en detalle.

PYnative.com

List in Python

```
L = [ 20, 'Jessa', 35.75, [30, 60, 90] ]
```

L[0]

L[1]

L[2]

L[3]

- ✓ **Ordered:** Maintain the order of the data insertion.
- ✓ **Changeable:** List is mutable and we can modify items.
- ✓ **Heterogeneous:** List can contain data of different types
- ✓ **Contains duplicate:** Allows duplicates data

Las siguientes son las propiedades de una lista.

1. **Mutable:** Los elementos de la lista pueden modificarse. Podemos añadir o eliminar elementos a la lista después de haberla creado.
2. **Ordenada:** Los elementos de las listas están ordenados. Cada elemento tiene un valor de índice único. Los nuevos elementos se añadirán al final de la lista.
3. **Heterogénea:** La lista puede contener diferentes tipos de elementos, es decir, puede contener elementos de cadena, enteros, booleanos o de cualquier tipo.

4. **Duplicados:** La lista puede contener duplicados, es decir, las listas pueden tener dos elementos con los mismos valores.

¿Por qué utilizar una lista?

La estructura de datos de lista es muy flexible. Tiene muchas funcionalidades únicas incorporadas, como `pop()`, `append()`, etc., que facilitan las cosas cuando los datos cambian constantemente.

Además, la lista puede contener elementos duplicados, es decir, dos o más elementos pueden tener los mismos valores.

Las listas son heterogéneas, es decir, se pueden añadir diferentes tipos de objetos/elementos

Como las listas son mutables, se utilizan en aplicaciones en las que los valores de los elementos cambian con frecuencia.

Creando listas en Python

La lista puede crearse utilizando el constructor de listas o utilizando corchetes `[]`.

- **Utilizando el constructor `list()`:** En general, el constructor de una clase tiene su nombre de clase. De forma similar, crea una lista pasando los valores separados por comas dentro de `list()`.
- **Usando corchetes `[]`:** En este método, podemos crear una lista simplemente encerrando los elementos dentro de los corchetes.

Vamos a verlo con algunos ejemplos

```
Python

# Using list constructor
my_list1 = list((1, 2, 3))
print(my_list1)

# Output [1, 2, 3]

# Using square brackets[]
my_list2 = [1, 2, 3]
```

```
print(my_list2)
# Output [1, 2, 3]

# with heterogeneous items
my_list3 = [1.0, 'Jessa', 3]
print(my_list3)
# Output [1.0, 'Jessa', 3]

# empty list using list()
my_list4 = list()
print(my_list4)
# Output []

# empty list using []
my_list5 = []
print(my_list4)
# Output []
```

Accediendo a elementos de la lista

Se puede acceder a los elementos de una lista mediante indexación y segmentación. Esta sección le guiará en el acceso a la lista utilizando las dos formas siguientes:

- **Utilizando la indexación**, podemos acceder a cualquier elemento de una lista utilizando su número de índice
- **Utilizando el slicing**, podemos acceder a un rango de elementos de una lista.

Indexación

Se puede acceder a los elementos de la lista mediante la técnica de «indexación». Las listas son colecciones ordenadas con índices únicos para cada elemento. Podemos acceder a los elementos de la lista utilizando este número de índice.

	P	Y	T	H	O	N	
Positive Indexing →	0	1	2	3	4	5	
	-6	-5	-4	-3	-2	-1	← Negative Indexing

Para acceder a los elementos de la lista de izquierda a derecha, se puede utilizar el valor del índice que empieza por cero hasta (longitud de la lista-1). Por ejemplo, si queremos acceder al 3er elemento debemos utilizar 2 ya que el valor del índice empieza en 0.

Nota:

- Como las Listas son secuencias ordenadas de elementos, los valores de índice comienzan desde 0 hasta la longitud de la Lista.
- Siempre que intentemos acceder a un elemento con un índice superior a la longitud de la lista, se producirá un error de índice.
- Del mismo modo, los índices son siempre enteros. Si indicamos cualquier otro tipo, se producirá un error de tipo.

Python

```
languages = ['Python', 'Swift', 'C++']  
  
# access the first element  
print('languages[0] =', languages[0])  
  
# access the third element
```

```
print('languages[2] =', languages[2])
```

Index Negativo

Se puede acceder a los elementos de la lista de derecha a izquierda utilizando la indexación negativa. El valor negativo comienza desde -1 hasta -longitud de la lista. Indica que la lista se indexa desde atrás/hacia atrás.

```
Python

my_list = [10, 20, 'Jessa', 12.50, 'Emma']

# accessing last element of the list
print(my_list[-1])

# output 'Emma'

# accessing second last element of the list
print(my_list[-2])

# output 12.5

# accessing 4th element from last
print(my_list[-4])

# output 20
```

List Slicing

Se puede acceder a los elementos de la lista de derecha a izquierda utilizando la indexación negativa. El valor negativo comienza desde -1 hasta -longitud de la lista. Indica que la lista se indexa desde atrás/hacia atrás.

Slicing es una operación de corte de una lista. Esto significa que puede extraer algunos elementos de una lista utilizando algunos índices preferidos. Permite seleccionar un rango de elementos de una lista creando una copia superficial de una lista que contenga sólo esos elementos.

L[start:stop:step]

Start position End position The increment

- El parámetro `start` indica la posición del índice a partir de la cual debe comenzar el corte y el parámetro `stop` indica las posiciones del índice hasta las que debe realizarse el corte.
- El parámetro `step` permite tomar cada n-ésimo elemento dentro de un rango `start_index:stop`

Diferentes combinaciones de Python

Todos los parámetros son opcionales. Si un parámetro no se establece explícitamente, Python utilizará un valor por defecto.

¿Cuáles son los valores por defecto de una rebanada python?

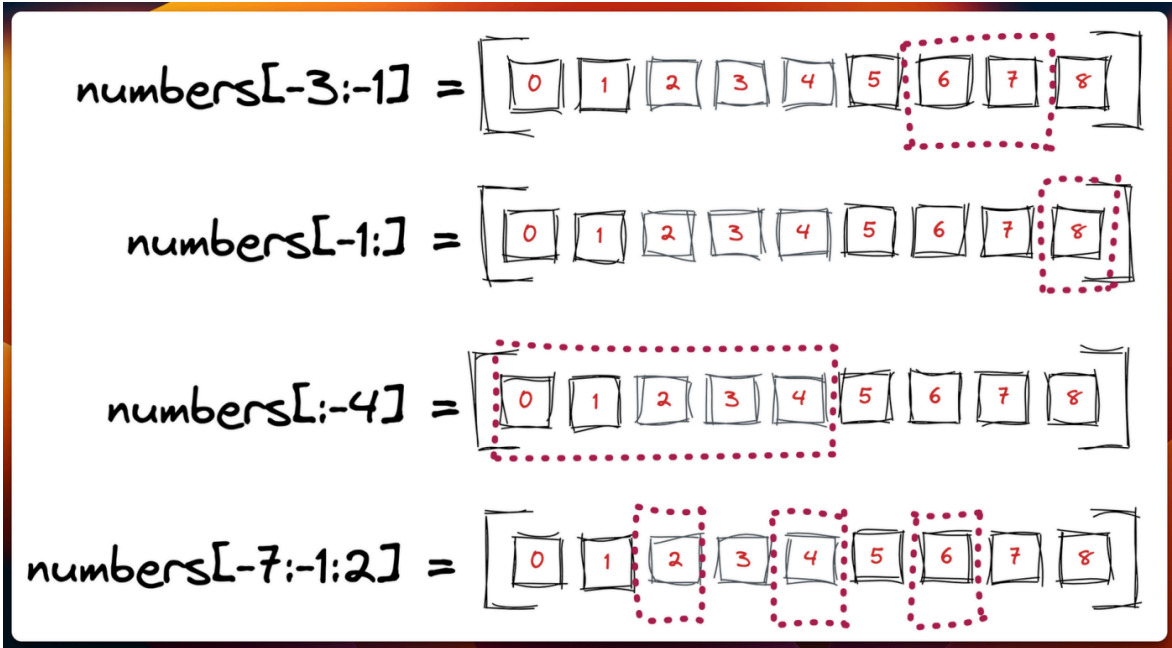
- Para `start` el valor por defecto es 0.
- Para `:stop` el valor por defecto es la longitud de la lista.
- Para `:step` el valor por defecto es 1

¿Qué valores se incluyen en el slice?

También puede preguntarse si el índice `:stop` se incluirá o no en el slice. La respuesta es no. La diferencia entre `start` y `:stop` es que el elemento del índice `start` se incluye en el corte, mientras que el elemento del índice `:stop` no.

Python slice negative index

¡Al utilizar python slices también puedes añadir índices negativos! ¿Qué significa esto? Bueno, significa que las listas en Python son un poco diferentes que digamos los arrays en otros lenguajes. El índice negativo es un índice relativo al final de la lista. El índice -1 es el último elemento de la lista. El índice -2 es el penúltimo elemento y así sucesivamente. Además, puede utilizar un valor negativo para la función de paso. Esto significa que la lista se recorre en la dirección opuesta. Hay más ejemplos a continuación para ayudarte a entender las peculiaridades del uso de slices.



Python

```
my_list = [0,1,2,3,4,5,6,7,8]
```

```
print(my_list[-3:-1])
```

```
print(my_list[-1:])
```

```
print(my_list[: -4])
```

```
print(my_list[-7:-1:2])
```

Output:

```
[6, 7]
[8]
[0, 1, 2, 3, 4]
[2, 4, 6]
```

Iterando una lista

Los objetos de la lista pueden iterarse uno a uno, utilizando un bucle for.

```
Python

my_list = [5, 8, 'Tom', 7.50, 'Emma']

# iterate a list

for item in my_list:

    print(item)
```

Resultado

Output:

```
5
8
Tom
7.5
Emma
```

Iterar junto con un número de índice

El valor del índice comienza desde 0 hasta (longitud de la lista-1). Por lo tanto, utilizar la función range() es ideal para este escenario.

La función range devuelve una secuencia de números. Por defecto, devuelve desde 0 hasta el número especificado (incrementa en 1). Los valores inicial y final pueden pasarse según nuestras necesidades.

Python

```
my_list = [5, 8, 'Tom', 7.50, 'Emma']

# iterate a list
for i in range(0, len(my_list)):

    # print each item using index number
    print(my_list[i])
```

Output:

5

8

Tom

7.5

Emma

Agregando elementos a la lista

Podemos añadir un nuevo elemento/lista de elementos a la lista utilizando los métodos de lista como **append()**, **insert()**, y **extend()**.

Añadir un elemento al final de la lista

El método **append()** sólo acepta un parámetro y lo añade al final de la lista.

Veamos el ejemplo para añadir el elemento “Emma” al final de la lista.

Python

```
my_list = list([5, 8, 'Tom', 7.50])
```



```
# Using append()

my_list.append('Emma')

print(my_list)

# Output [5, 8, 'Tom', 7.5, 'Emma']


# append the nested list at the end

my_list.append([25, 50, 75])

print(my_list)

# Output [5, 8, 'Tom', 7.5, 'Emma', [25, 50, 75]]
```

Añadir elemento en la posición especificada de la lista

Utilice el método `insert()` para añadir el objeto/elemento en la posición especificada de la lista. El método `insert` acepta dos parámetros: `position` y `object`.

```
insert(index, object)
```

```
Python

my_list = list([5, 8, 'Tom', 7.50])


# Using insert()

# insert 25 at position 2

my_list.insert(2, 25)

print(my_list)

# Output [5, 8, 25, 'Tom', 7.5]


# insert nested list at at position 3
```

```
my_list.insert(3, [25, 50, 75])  
  
print(my_list)
```

Usando extend()

El método extend aceptará la lista de elementos y los añadirá al final de la lista. Incluso podemos añadir otra lista utilizando este método.

Vamos a añadir tres elementos al final de la lista.

```
Python  
  
my_list = list([5, 8, 'Tom', 7.50])  
  
# Using extend()  
my_list.extend([25, 75, 100])  
  
print(my_list)  
  
Output:  
  
[5, 8, 'Tom', 7.5, 25, 75, 100]
```

Modificando los elementos de la lista

La lista es una secuencia mutable de objetos iterables. Esto significa que podemos modificar los elementos de una lista. Utilice el número de índice y el operador de asignación (=) para asignar un nuevo valor a un elemento.

Veamos cómo realizar los dos siguientes escenarios de modificación

- Modificar el elemento individual.

- Modificar el rango de elementos

Python

```
my_list = list([2, 4, 6, 8, 10, 12])
```

```
# modify single item
```

```
my_list[0] = 20
```

```
print(my_list)
```

```
# Output [20, 4, 6, 8, 10, 12]
```

```
# modify range of items
```

```
# modify from 1st index to 4th
```

```
my_list[1:4] = [40, 60, 80]
```

```
print(my_list)
```

```
# Output [20, 40, 60, 80, 10, 12]
```

```
# modify from 3rd index to end
```

```
my_list[3:] = [80, 100, 120]
```

```
print(my_list)
```

```
# Output [20, 40, 60, 80, 100, 120]
```

Eliminando los elementos de la lista

method	Description
<code>remove(item)</code>	To remove the first occurrence of the item from the list.
<code>pop(index)</code>	Removes and returns the item at the given index from the list.
<code>clear()</code>	To remove all items from the list. The output will be an empty list.
<code>del list_name</code>	Delete the entire list.

Eliminando un elemento específico

Utilice el método `remove()` para eliminar la primera aparición del elemento de la lista.

Nota: Lanza un `keyerror` si un elemento no está presente en la lista original.

```
Python

my_list = list([2, 4, 6, 8, 10, 12])

# remove item 6
my_list.remove(6)

# remove item 8
my_list.remove(8)

print(my_list)

# Output [2, 4, 10, 12]
```

Eliminar el elemento presente en el índice dado

Utilice el método `pop()` para eliminar el elemento presente en el índice dado. El método `pop()` elimina y devuelve el elemento presente en el índice dado.

Nota: Eliminará el último elemento de la lista si no se indica el número de índice.

Python

```
my_list = list([2, 4, 6, 8, 10, 12])

# remove item present at index 2
my_list.pop(2)
print(my_list)
# Output [2, 4, 8, 10, 12]

# remove item without passing index number
my_list.pop()
print(my_list)
# Output [2, 4, 8, 10]
```

Eliminar el rango de elementos

Utilice la palabra clave `del` junto con el corte de lista para eliminar el rango de elementos.

Python

```
my_list = list([2, 4, 6, 8, 10, 12])

# remove range of items
# remove item from index 2 to 5
del my_list[2:5]
print(my_list)
# Output [2, 4, 12]
```

```
# remove all items starting from index 3

my_list = list([2, 4, 6, 8, 10, 12])

del my_list[3:]

print(my_list)
```

Eliminar todos los elementos

Utilice el método `clear()` de la lista para eliminar todos los elementos de la lista. El método `clear()` trunca la lista.

```
Python

my_list = list([2, 4, 6, 8, 10, 12])

# clear list

my_list.clear()

print(my_list)

# Output []

# Delete entire list

del my_list
```

Encontrar un elemento en la lista

Utilice la función `index()` para encontrar un elemento en una lista.

La función `index()` aceptará el valor del elemento como parámetro y devuelve la primera aparición del elemento o devuelve `ValueError` si el elemento no existe.

Python

```
my_list = list([2, 4, 6, 8, 10, 12])

print(my_list.index(8))

# Output 3

# returns error since the element does not exist in the list.

# my_list.index(100)
```

Concatenación de dos listas

La concatenación de dos listas significa la fusión de dos listas. Hay dos formas de hacerlo

Utilizando el operador +.

Utilizando el método `extend()`. El método `extend()` añade los elementos de la nueva lista al final de la lista llamada.

Python

```
my_list1 = [1, 2, 3]
my_list2 = [4, 5, 6]

# Using + operator

my_list3 = my_list1 + my_list2

print(my_list3)

# Output [1, 2, 3, 4, 5, 6]
```

```
# Using extend() method

my_list1.extend(my_list2)

print(my_list1)

# Output [1, 2, 3, 4, 5, 6]
```

Copia de una lista

Existen dos formas de crear una copia de una lista. Veamos cada una con un ejemplo.

Utilizando el operador de asignación (=)

Esta es una forma directa de crear una copia. En este método, la nueva lista será una copia profunda. Los cambios que hagamos en la lista original se reflejarán en la nueva lista.

Esto se llama copia profunda (deep coding)

```
Python

my_list1 = [1, 2, 3]

# Using = operator

new_list = my_list1

# printing the new list

print(new_list)

# Output [1, 2, 3]

# making changes in the original list

my_list1.append(4)
```



```
# print both copies
print(my_list1)
# result [1, 2, 3, 4]
print(new_list)
# result [1, 2, 3, 4]
```

Como se ve en el ejemplo anterior, se ha creado una copia de la lista. Los cambios realizados en la lista original se reflejan también en la lista copiada.

Nota: Cuando estableces `lista1 = lista2`, estás haciendo que se refieran al mismo objeto lista, por lo que cuando modificas uno de ellos, todas las referencias asociadas a ese objeto reflejan el estado actual del objeto. Así que no utilice el operador de asignación para copiar el diccionario, en su lugar utilice el método `copy()`.

Uso del método `copy()`

El método `copy` permite crear una copia de una lista. Esto crea una nueva lista y los cambios realizados en la lista original no se reflejan en la nueva. Esto se conoce como copia superficial (shallow copy)

```
Python
my_list1 = [1, 2, 3]

# Using copy() method
new_list = my_list1.copy()

# printing the new list
print(new_list)
```

```
# Output [1, 2, 3]

# making changes in the original list
my_list1.append(4)

# print both copies
print(my_list1)
# result [1, 2, 3, 4]
print(new_list)
# result [1, 2, 3]
```

Como se ve en el ejemplo anterior, se ha creado una copia de la lista. Los cambios realizados en la lista original no se reflejan en la copia.

Operaciones con listas

Podemos realizar operaciones con la lista mediante funciones como `sort()`, `reverse()`, `clear()`, etc.

Ordenar la lista con `sort()`

La función `sort` ordena los elementos de la lista en orden ascendente.

Python

```
mylist = [3,2,1]
mylist.sort()
print(mylist)
```

Invertir una lista usando reverse()

La función reverse se utiliza para invertir los elementos de la lista.

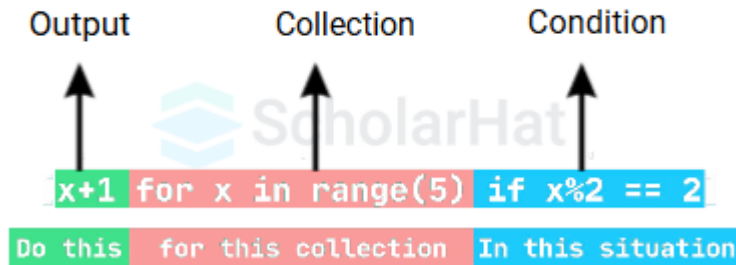
Python

```
mylist = [3, 4, 5, 6, 1]
mylist.reverse()
print(mylist)
```

List Comprehension

La comprensión de listas es un método más sencillo para crear una lista a partir de una lista existente. Generalmente, se trata de una lista de iterables generada con la opción de incluir solo los elementos que cumplen una condición.

LIST COMPREHENSION



Entendamos la comprensión de listas en Python a través de algunos puntos:

- Usando la comprensión de listas en Python, puedes generar fácilmente una nueva lista aplicando una expresión a cada elemento de una lista iterable o existente.
- Permite crear código claro y simple en una sola línea, a diferencia de las varias líneas llenas de bucles de Python.
- También puedes agregar condiciones para filtrar los elementos que deben incluirse en la nueva lista.
- La comprensión de listas en Python es más rápida y eficiente que los bucles tradicionales para crear listas.
- Es ideal para transformar datos, como elevar números al cuadrado, filtrar números pares o impares, o manipular cadenas.

Python

```
inputList = [4,7,11,13,18,20]

#creating a list with square values of only the even numbers

squareList = [var**2 for var in inputList if var%2==0]

print(squareList)
```

#Output:

```
#[16, 324, 400]
```

Python

```
# Using list comprehension to create a list of the first ten even numbers
```

```
even_numbers = [x for x in range(1, 21) if x % 2 == 0]
```

```
print(even_numbers)
```

```
# Output:
```

```
# [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

Tuplas en Python

En esta sesión, aprenderás a utilizar una estructura de datos de tupla en Python. Además, aprenderás cómo crear, acceder y modificar una tupla en Python y todas las demás operaciones que podemos realizar sobre una tupla.

Qué es una tupla

Las tuplas son colecciones ordenadas de datos heterogéneos que son inmutables. Heterogéneo significa que la tupla puede almacenar variables de todo tipo.

Las tuplas tienen las siguientes características

- **Ordenadas:** Las tuplas forman parte de los tipos de datos secuenciales, lo que significa que mantienen el orden de inserción de los datos. Mantiene el valor del índice para cada elemento.
- **Inalterables:** Las tuplas son inmutables, lo que significa que no podemos añadir o eliminar elementos a la tupla después de su creación.
- **Heterogéneas:** Las tuplas son una secuencia de datos de diferentes tipos de datos (como entero, flotante, lista, cadena, etc;) y se puede acceder a ellas mediante indexación y segmentación.
- **Contiene duplicados:** Las tuplas pueden contener duplicados, lo que significa que pueden tener elementos con el mismo valor.

Tuples in Python

```
T = ( 20, 'Jessa', 35.75, [30, 60, 90] )
```

↑ ↑ ↑ ↑
T[0] T[1] T[2] T[3]

- ✓ **Ordered:** Maintain the order of the data insertion.
- ✓ **Unchangeable:** Tuples are immutable and we can't modify items.
- ✓ **Heterogeneous:** Tuples can contains data of types
- ✓ **Contains duplicate:** Allows duplicates data

Creación de una tupla

Podemos crear una tupla de dos maneras

- **Utilizando paréntesis ():** Una tupla se crea encerrando elementos separados por comas dentro de paréntesis redondeados.
- **Utilizando un constructor tuple():** Se crea una tupla pasando los elementos separados por comas dentro de tuple().

Python

```
# create a tuple using ()

# number tuple
number_tuple = (10, 20, 25.75)
print(number_tuple)

# Output (10, 20, 25.75)


# string tuple
string_tuple = ('Jessa', 'Emma', 'Kelly')
print(string_tuple)

# Output ('Jessa', 'Emma', 'Kelly')


# mixed type tuple
sample_tuple = ('Jessa', 30, 45.75, [25, 78])
print(sample_tuple)

# Output ('Jessa', 30, 45.75, [25, 78])


# create a tuple using tuple() constructor
sample_tuple2 = tuple(('Jessa', 30, 45.75, [23, 78]))
print(sample_tuple2)

# Output ('Jessa', 30, 45.75, [23, 78])
```

Crear una tupla con un único elemento

Una tupla con un único elemento se crea encerrando un elemento entre paréntesis seguido de una coma. Si la tupla tiempo es una cadena encerrada entre paréntesis y no seguida de una coma, Python la trata como un tipo str. Veámoslo con un ejemplo.

Python

```
# without comma

single_tuple = ('Hello')

print(type(single_tuple))

# Output class 'str'

print(single_tuple)

# Output Hello


# with comma

single_tuple1 = ('Hello',)

# output class 'tuple'

print(type(single_tuple1))

# Output ('Hello',)

print(single_tuple1)
```

Empaquetado y desempaquetado

Una tupla también se puede crear sin usar un constructor tuple() ni encerrar los elementos entre paréntesis. Esto se denomina variable "Empaquetado".

Packing and Unpacking

En Python, podemos crear una tupla empaquetando un grupo de variables. El empaquetado se puede utilizar cuando queremos recopilar múltiples valores en una sola variable. Generalmente, esta operación se conoce como empaquetado de tuplas.

De igual forma, podemos desempaquetar los elementos simplemente asignándole al mismo número de variables. Este proceso se denomina "Desempaquetado".

Veámoslo con un ejemplo.

Python

```
# packing variables into tuple

tuple1 = 1, 2, "Hello"

# display tuple

print(tuple1)

# Output (1, 2, 'Hello')


print(type(tuple1))

# Output class 'tuple'


# unpacking tuple into variable

i, j, k = tuple1

# printing the variables

print(i, j, k)

# Output 1 2 Hello
```

Como podemos ver en el resultado anterior, tres elementos de la tupla están asignados a las variables i, j y k, respectivamente.

Si asignamos menos variables que elementos en la tupla, obtendremos un error de valor con el mensaje "demasiados valores para descomprimir".

Longitud de una tupla

Podemos calcular la longitud de una tupla mediante la función `len()`. Esta función devolverá el número de elementos de la tupla.

Python

```
tuple1 = ('P', 'Y', 'T', 'H', 'O', 'N')  
  
# length of a tuple  
print(len(tuple1))  
  
# Output 6
```

Iterar una tupla

Podemos iterar una tupla usando un bucle for. Veámoslo con un ejemplo.

Python

```
# create a tuple  
sample_tuple = tuple((1, 2, 3, "Hello", [4, 8, 16]))  
  
# iterate a tuple  
  
for item in sample_tuple:  
    print(item)
```

Acceso a los elementos de una tupla

Se puede acceder a una tupla mediante indexación y segmentación. Esta sección le guiará para acceder a una tupla de dos maneras:

- Mediante la indexación, podemos acceder a cualquier elemento de una tupla utilizando su número de índice.
- Mediante la segmentación, podemos acceder a un rango de elementos de una tupla.

Indexación

Una tupla es una secuencia ordenada de elementos, lo que significa que mantiene el orden de inserción de datos. Mantiene el valor del índice de cada elemento.

Podemos acceder a un elemento de una tupla utilizando su número de índice dentro del operador de índice []; este proceso se denomina "indexación".

Nota:

Como las tuplas son secuencias ordenadas de elementos, los valores del índice van desde 0 hasta la longitud de la tupla.

Siempre que intentemos acceder a un elemento con un índice mayor que la longitud de la tupla, se generará un error de índice.

De igual forma, los valores del índice siempre son enteros. Si asignamos cualquier otro tipo, se generará un error de tipo.

	P	Y	T	H	O	N
Positive Indexing →	0	1	2	3	4	5
	-6	-5	-4	-3	-2	-1
						← Negative Indexing

En la imagen de arriba, podemos ver que los valores del índice comienzan desde cero y continúan hasta el último elemento cuyo valor de índice será `len(tuple) - 1`.

Python

```
tuple1 = ('P', 'Y', 'T', 'H', 'O', 'N')  
  
for i in range(4):  
    print(tuple1[i])
```

Output:

P
Y
T
H



Como se ve en el ejemplo anterior, imprimimos los primeros cuatro elementos de la tupla con el índice.

Nota: Si mencionamos que el valor del índice es mayor que la longitud de una tupla, se generará un error de índice.

Diccionarios en Python

Los diccionarios son colecciones ordenadas de valores únicos almacenados en pares clave-valor. Un diccionario de Python representa la correspondencia entre una clave y un valor. En resumen, un diccionario de Python puede almacenar pares de claves y valores. Cada clave está vinculada a un valor específico. Una vez almacenada en un diccionario, se puede obtener el valor utilizando únicamente la clave.

Por ejemplo, considere la búsqueda de teléfono, donde es muy fácil y rápido encontrar el número de teléfono (valor) cuando conocemos el nombre (clave) asociado.

Dictionary in Python PYnative.com

Unordered collections of unique values stored in (Key-Value) pairs.

```
d = {'a': 10, 'b': 20, 'c': 30}
```

↑
d['a']

↑
d['b']

↑
d['c']

- ✓ **Unordered:** The items in dict are stored without any index value
- ✓ **Unique:** Keys in dictionaries should be Unique
- ✓ **Mutable:** We can add/Modify/Remove key-value after the creation

- **Desordenado (en Python 3.6 y versiones anteriores):** Los elementos de los diccionarios se almacenan sin ningún valor de índice, que suele ser un rango de números. Se almacenan como pares clave-valor, y las claves son su índice, que no seguirá ninguna secuencia.
- **Ordenado (en Python 3.7 y versiones posteriores):** Los diccionarios están ordenados, lo que significa que los elementos tienen un orden definido que no cambia. Una tabla hash simple consiste en pares clave-valor organizados en orden pseudoaleatorio según los cálculos de la función hash.
- **Único:** Como se mencionó anteriormente, cada valor tiene una clave; las claves de los diccionarios deben ser únicas. Si almacenamos cualquier valor con una clave ya existente, el valor más reciente reemplazará al anterior.

- **Mutable:** Los diccionarios son colecciones modificables, lo que implica que podemos añadir o eliminar elementos después de su creación.

Creación de un Diccionario

Hay tres maneras de crear un diccionario.

1. Usando llaves: Los diccionarios se crean encerrando los pares clave-valor separados por comas dentro de las llaves {}. Los dos puntos «:» se usan para separar la clave y el valor en un par.
2. Usando el constructor dict(): Crea un diccionario pasando los pares clave-valor separados por comas dentro de dict().
3. Usando una secuencia que tiene cada elemento como un par (clave-valor).

Veamos cada uno de ellos con un ejemplo.

Python

```
# create a dictionary using {}  
person = {"name": "Jessa", "country": "USA", "telephone": 1178}  
print(person)  
  
# output {'name': 'Jessa', 'country': 'USA', 'telephone': 1178}  
  
# create a dictionary using dict()  
person = dict({"name": "Jessa", "country": "USA", "telephone": 1178})  
print(person)  
  
# output {'name': 'Jessa', 'country': 'USA', 'telephone': 1178}  
  
# create a dictionary from sequence having each item as a pair  
person = dict([("name", "Mark"), ("country", "USA"), ("telephone", 1178)])  
print(person)  
  
# create dictionary with mixed keys keys
```

```
# first key is string and second is an integer

sample_dict = {"name": "Jessa", 10: "Mobile"}

print(sample_dict)

# output {'name': 'Jessa', 10: 'Mobile'}


# create dictionary with value as a list

person = {"name": "Jessa", "telephones": [1178, 2563, 4569]}

print(person)

# output {'name': 'Jessa', 'telephones': [1178, 2563, 4569]}
```

Diccionario vacío

Cuando creamos un diccionario sin ningún elemento dentro de las llaves, será un diccionario vacío.

```
Python

emptydict = {}

print(type(emptydict))

# Output class 'dict'
```

Acceso a los elementos de un diccionario

Hay dos maneras de acceder a los elementos de un diccionario.

1. Recuperar el valor usando el nombre de la clave entre corchetes [].
2. Recuperar el valor pasando el nombre de la clave como parámetro al método get() de un diccionario.

Python

```
# create a dictionary named person
```

```
person = {"name": "Jessa", "country": "USA", "telephone": 1178}
```

```
# access value using key name in []
```

```
print(person['name'])
```

```
# Output 'Jessa'
```

```
# get key value using key name in get()
```

```
print(person.get('telephone'))
```

```
# Output 1178
```

Obtener todas las claves y valores

Utilice los siguientes métodos de diccionario para recuperar todas las claves y valores a la vez.

Method	Description
<code>keys()</code>	Returns the list of all keys present in the dictionary.
<code>values()</code>	Returns the list of all values present in the dictionary
<code>items()</code>	Returns all the items present in the dictionary. Each item will be inside a tuple as a key-value pair.

Python

```
person = {"name": "Jessa", "country": "USA", "telephone": 1178}

# Get all keys
print(person.keys())

# output dict_keys(['name', 'country', 'telephone'])
print(type(person.keys()))

# Output class 'dict_keys'

# Get all values
print(person.values())

# output dict_values(['Jessa', 'USA', 1178])
print(type(person.values()))

# Output class 'dict_values'

# Get all key-value pair
print(person.items())

# output dict_items([('name', 'Jessa'), ('country', 'USA'), ('telephone', 1178)])
print(type(person.items()))

# Output class 'dict_items'
```

Iteración de un diccionario

Podemos iterar un diccionario mediante un bucle for y acceder a las claves individuales y sus valores correspondientes. Veamos esto con un ejemplo.

Python

```
person = {"name": "Jessa", "country": "USA", "telephone": 1178}

# Iterating the dictionary using for-loop
print('key', ':', 'value')
for key in person:
    print(key, ':', person[key])

# using items() method
print('key', ':', 'value')
for key_value in person.items():
    # first is key, and second is value
    print(key_value[0], key_value[1])
```

Output:

```
key : value
name : Jessa
country : USA
telephone : 1178
key : value
name Jessa
country USA
telephone 1178
```

Añadiendo elementos al diccionario

Podemos añadir nuevos elementos al diccionario de dos maneras:

1. Mediante la asignación de clave-valor: Mediante una simple sentencia de asignación, el valor se puede asignar directamente a la nueva clave.

2. Mediante el método `update()`: En este método, el elemento pasado dentro del método `update()` se insertará en el diccionario. El elemento puede ser otro diccionario o cualquier iterable, como una tupla de pares clave-valor.

Python

```
person = {"name": "Jessa", 'country': "USA", "telephone": 1178}

# update dictionary by adding 2 new keys
person["weight"] = 50
person.update({"height": 6})

# print the updated dictionary
print(person)

# output {'name': 'Jessa', 'country': 'USA', 'telephone': 1178, 'weight': 50, 'height': 6}
```

Python

```
person = {"name": "Jessa", 'country': "USA"}

# Adding 2 new keys at once
# pass new keys as dict
person.update({"weight": 50, "height": 6})

# print the updated dictionary
print(person)

# output {'name': 'Jessa', 'country': 'USA', 'weight': 50, 'height': 6}
```

```
# pass new keys as as list of tuple
person.update([("city", "Texas"), ("company", "Google",)])

# print the updated dictionary
print(person)
```

Modificar los valores de las claves del diccionario

Podemos modificar los valores de las claves del diccionario existentes de dos maneras:

- Usando el nombre de la clave: Podemos asignar nuevos valores directamente usando su nombre de clave. El nombre de la clave será el existente y podemos mencionar el nuevo valor.
- Usando el método update(): Podemos usar el método update pasando el par clave-valor para cambiar el valor. En este caso, el nombre de la clave será el existente y el valor a actualizar será el nuevo.

Python

```
person = {"name": "Jessa", "country": "USA"}

# updating the country name
person["country"] = "Canada"

# print the updated country
print(person['country'])

# Output 'Canada'

# updating the country name using update() method
```

```
person.update({"country": "USA"})

# print the updated country

print(person['country'])
```

Eliminar elementos del diccionario

Existen varios métodos para eliminar elementos del diccionario. Tanto si queremos eliminar un solo elemento, el último insertado o todo el diccionario, podemos elegir el método que queramos utilizar.

Utilice los siguientes métodos para eliminar claves de un diccionario.

Method	Description
<code>pop(key[,d])</code>	Return and removes the item with the <code>key</code> and return its value. If the <code>key</code> is not found, it raises <code>KeyError</code> .
<code>popitem()</code>	Return and removes the last inserted item from the dictionary. If the dictionary is empty, it raises <code>KeyError</code> .
<code>del key</code>	The <code>del</code> keyword will delete the item with the key that is passed
<code>clear()</code>	Removes all items from the dictionary. Empty the dictionary
<code>del dict_name</code>	Delete the entire dictionary

Python

```
person = {'name': 'Jessa', 'country': 'USA', 'telephone': 1178, 'weight': 50, 'height': 6}
```

```

# Remove last inserted item from the dictionary
deleted_item = person.popitem()
print(deleted_item) # output ('height', 6)

# display updated dictionary
print(person)

# Output {'name': 'Jessa', 'country': 'USA', 'telephone': 1178, 'weight': 50}


# Remove key 'telephone' from the dictionary
deleted_item = person.pop('telephone')
print(deleted_item) # output 1178

# display updated dictionary
print(person)

# Output {'name': 'Jessa', 'country': 'USA', 'weight': 50}


# delete key 'weight'
del person['weight']

# display updated dictionary
print(person)

# Output {'name': 'Jessa', 'country': 'USA'}


# remove all item (key-values) from dict
person.clear()

# display updated dictionary
print(person) # {}


# Delete the entire dictionary
del person

```

Comprobación de la existencia de una clave

Para comprobar si una clave específica existe en un diccionario, podemos usar el método `keys()` y el operador `in`. Podemos usar el operador `in` para comprobar si la clave está presente en la lista de claves devueltas por el método `keys()`.

Con este método, podemos comprobar si nuestra clave está presente en la lista de claves que devolverá el método `keys()`.

Comprobemos si la clave "country" existe e imprimimos su valor si se encuentra.

```
Python

person = {'name': 'Jessa', 'country': 'USA', 'telephone': 1178}

# Get the list of keys and check if 'country' key is present
key_name = 'country'
if key_name in person.keys():
    print("country name is", person[key_name])
else:
    print("Key not found")
# Output country name is USA
```

Unir dos diccionarios

Podemos añadir dos diccionarios mediante el método `update()` o descomprimiendo palabras clave arbitrarias con el operador `**`. Veamos cada caso con un ejemplo.

Uso del método `update()`

En este método, el diccionario que se añadirá se pasará como argumento al método `update()` y el diccionario actualizado contendrá elementos de ambos diccionarios.

Veamos cómo fusionar el segundo diccionario con el primero.

Python

```
dict1 = {'Jessa': 70, 'Arul': 80, 'Emma': 55}
dict2 = {'Kelly': 68, 'Harry': 50, 'Olivia': 66}

# copy second dictionary into first dictionary
dict1.update(dict2)

# printing the updated dictionary
print(dict1)

# output {'Jessa': 70, 'Arul': 80, 'Emma': 55, 'Kelly': 68, 'Harry': 50, 'Olivia': 66}
```

Como se ve en el ejemplo anterior, los elementos de ambos diccionarios se han actualizado y el `dict1` contendrá los elementos de ambos.

Uso de ****kwargs** para descomprimir

Podemos descomprimir cualquier número de diccionarios y añadir su contenido a otro diccionario usando ****kwargs**. De esta forma, podemos añadir múltiples argumentos de longitud a un diccionario con una sola instrucción.

Python

```
student_dict1 = {'Aadya': 1, 'Arul': 2, }
student_dict2 = {'Harry': 5, 'Olivia': 6}
student_dict3 = {'Nancy': 7, 'Perry': 9}
```

```
# join three dictionaries

student_dict = {**student_dict1, **student_dict2, **student_dict3}

# printing the final Merged dictionary

print(student_dict)

# Output {'Aadya': 1, 'Arul': 2, 'Harry': 5, 'Olivia': 6, 'Nancy': 7,
'Perry': 9}
```

Decoradores en Python

Los decoradores son una característica potente y elegante de Python que permite modificar o ampliar el comportamiento de funciones y métodos sin cambiar su código.

Un decorador es un patrón de diseño en Python que permite al usuario añadir nuevas funcionalidades a un objeto existente sin modificar su estructura. Los decoradores se aplican típicamente a las funciones, y juegan un papel crucial en la mejora o modificación del comportamiento de las funciones. Tradicionalmente, los decoradores se colocan antes de la definición de una función que se desea decorar. En este tutorial, demostraremos cómo utilizar eficazmente los decoradores en las funciones de Python.

Funciones como objetos de primera clase

Las funciones en Python son **first-class citizens**. Esto significa que soportan operaciones como ser pasadas como argumento, devueltas desde una función, modificadas y asignadas a una variable. Esta propiedad es crucial ya que permite que las funciones sean tratadas como cualquier otro objeto en Python, permitiendo una mayor flexibilidad en la programación. En conclusión, las funciones de primera clase significan que las funciones en Python son tratadas como cualquier otro objeto. Esto implica que las funciones pueden ser:

- Pasadas como argumentos a otras funciones.
- Devueltas por otras funciones.
- Asignadas a variables.

Asignación de funciones a variables

Para empezar crearemos una función que sumará uno a un número cada vez que sea llamada. A continuación, asignaremos la función a una variable y utilizaremos esta variable para llamar a la función.

Python

```
def plus_one(number):  
    return number + 1  
  
add_one = plus_one  
print(add_one(5))  
  
#6
```

Definir funciones dentro de otras funciones

Definir funciones dentro de otras funciones es una potente característica de Python, y es esencial para construir decoradores. Veamos otra idea central: pasar funciones como argumentos. Esto nos llevará un paso más cerca de escribir decoradores.

Python

```
def plus_one(number):  
    def add_one(number):  
        return number + 1  
  
    result = add_one(number)  
    return result  
  
value = plus_one(4)  
print(value)  
  
#6
```

Pasar funciones como argumentos a otras funciones

Las funciones también pueden pasarse como parámetros a otras funciones. Ilustrémoslo a continuación.

Python

```
def plus_one(number):  
    return number + 1  
  
def function_call(function):  
    number_to_add = 5  
    return function(number_to_add)  
  
value = function_call(plus_one)  
print(value)  
#6
```

Funciones que devuelven otras funciones

Una función también puede generar otra función. Lo mostraremos a continuación con un ejemplo.

Python

```
def hello_function():  
    def say_hi():  
        return "Hi"
```

```
    return say_hi

hello = hello_function()

print(hello())

#Hi
```

Funciones internas y cierres

Python permite que una función anidada acceda al ámbito externo de la función que la encierra. Este es un concepto crítico en los decoradores, conocido como cierre.

Un cierre en Python es una función que recuerda el entorno en el que fue creada, incluso después de que ese entorno ya no esté activo. Esto significa que una función anidada puede «cerrar sobre» variables de su ámbito envolvente y continuar usándolas.

Los cierres son esenciales para entender los decoradores porque los decoradores se basan en la capacidad de una función envolvente anidada para acceder y modificar el estado de la función decoradora que la encierra.

```
Python

def outer_function(message):

    def inner_function():

        print(f"Message from closure: {message}")

    return inner_function

closure_function = outer_function("Hello, closures!")

closure_function()

# Output: Message from closure: Hello, closures!
```

En este ejemplo:

- `inner_function` es un cierre porque accede a un mensaje, una variable de su ámbito (`outter_function`).
- Aunque la `outer_function` haya terminado de ejecutarse, la `inner_function` conserva el acceso al mensaje.

Cuando se crea un decorador, la función envolvente (dentro del decorador) es un cierre. Conserva el acceso a la función decorada y a cualquier estado o argumento adicional definido en la función decoradora. Por ejemplo:

```
Python

def simple_decorator(func):
    def wrapper():
        print("Before the function call")
        func()
        print("After the function call")
    return wrapper

@simple_decorator
def greet():
    print("Hello!")

greet()

# Output:
# Before the function call
# Hello!
# After the function call
```

Aquí, `wrapper` es un cierre que recuerda el saludo de la función y añade comportamiento antes y después de su ejecución.

Creación de su primer decorador

Ahora que ya conoce los cierres (la capacidad de una función para recordar variables de su ámbito), estamos listos para crear nuestro primer decorador real. Los cierres son la salsa secreta que permite a los decoradores trabajar entre bastidores.

Vamos a crear un decorador sencillo que convierta una sentencia a mayúsculas. Hacemos esto definiendo una envoltura dentro de una función cerrada. Como puedes ver es muy similar a la función dentro de otra función que creamos anteriormente.

Python

```
def uppercase_decorator(function):  
    def wrapper():  
        func = function()  
        make_uppercase = func.upper()  
        return make_uppercase  
  
    return wrapper
```

Como nuestro decorador toma una función como argumento, definiremos una nueva función y se la pasaremos al decorador. Antes aprendimos que podíamos asignar una función a una variable. Usaremos ese truco para llamar a la función de nuestro decorador.

Python

```
def say_hi():  
    return 'hello there'  
  
decorate = uppercase_decorator(say_hi)  
print(decorate())  
  
#HELLO THERE
```


Usando la sintaxis @

Sin embargo, Python nos proporciona una forma mucho más sencilla de aplicar decoradores. Simplemente usamos el símbolo @ antes de la función que queremos decorar. Veámoslo en la práctica a continuación.

Python

```
@uppercase_decorator
```

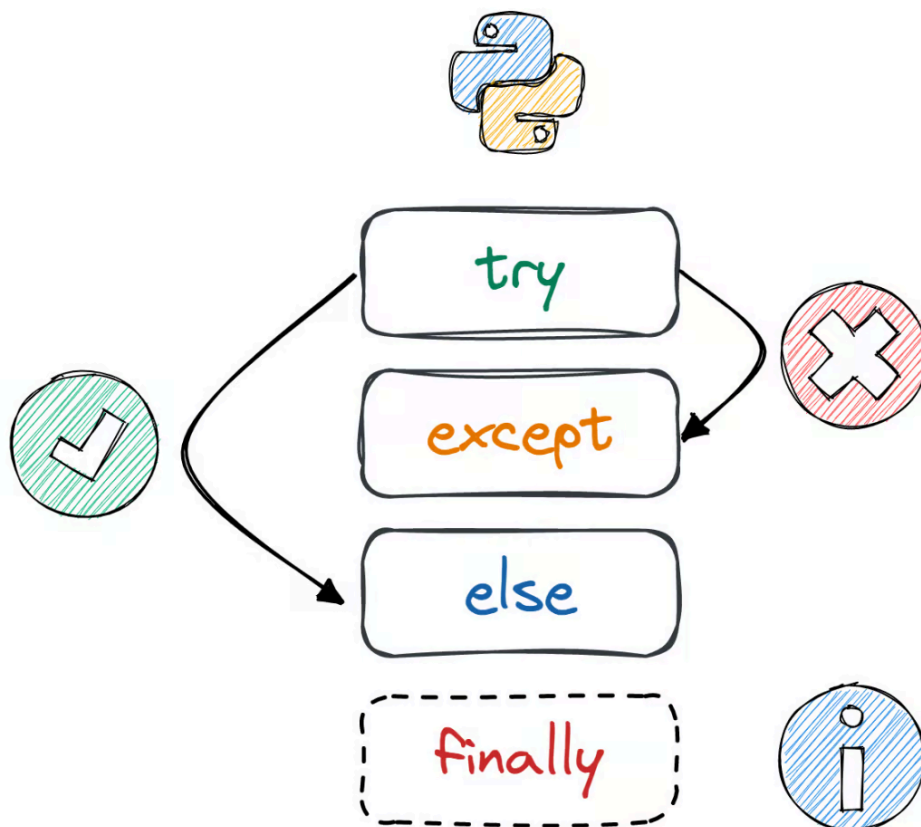
```
def say_hi():
```

```
    return 'hello there'
```

```
say_hi()
```

Manejo de Excepciones en Python

Los errores y las excepciones pueden provocar fallas en el programa o un comportamiento inesperado, y Python viene con un sólido conjunto de herramientas para mejorar la estabilidad del código.



Los errores y las excepciones pueden provocar un comportamiento inesperado o incluso detener la ejecución de un programa. Python ofrece diversas funciones y mecanismos para gestionar estos problemas y mejorar la robustez del código. En este tutorial, aprenderemos sobre los distintos tipos de errores y las funciones integradas con ejemplos.

Un error es un problema en un programa que impide que este complete su tarea. En comparación, una excepción es una condición que interrumpe el flujo normal del programa. Tanto los errores como las excepciones son un tipo de error en tiempo de ejecución, lo que significa que ocurren durante la ejecución de un programa.

En resumen, un error es un problema crítico que una aplicación normal no debería detectar, mientras que una excepción es una condición que un programa sí debería detectar.

Errores en Python

Aquí hay un ejemplo de un error de sintaxis donde un retorno fuera de la función no significa nada. No debemos gestionar errores en un programa. En su lugar, debemos crear una función que devuelva la cadena.

Python

```
return "Mensaje de prueba"
```

Output:

File "/main.py", line 1

```
    return "DataCamp"
```

```
^^^^^^^^^^^^^^^^^^
```

SyntaxError: 'return' outside function

Python

Input In [1]

```
return "Mensaje de prueba"
```

```
^
```

SyntaxError: 'return' outside function

Excepciones en Python

Python utiliza un objeto especial llamado excepción para controlar cualquier error que pueda ocurrir durante la ejecución de un programa.

Cuando ocurre un error durante la ejecución de un programa, Python crea una excepción. Si no se controla esta excepción la ejecución del programa se detiene y se muestra el error (traceback).

Python

```
print(1 / 0) # Error al intentar dividir por 0.
```

Traceback (most recent call last):

```
File "<stdin>", line 1, in <module>
```

ZeroDivisionError: division by zero

Tipos de excepciones en Python

Los principales excepciones definidas en Python son:

- **TypeError** : Ocurre cuando se aplica una operación o función a un dato del tipo inapropiado.
- **ZeroDivisionError** : Ocurre cuando se intenta dividir por cero.
- **OverflowError** : Ocurre cuando un cálculo excede el límite para un tipo de dato numérico.
- **IndexError** : Ocurre cuando se intenta acceder a una secuencia con un índice que no existe.
- **KeyError** : Ocurre cuando se intenta acceder a un diccionario con una clave que no existe.
- **FileNotFoundError** : Ocurre cuando se intenta acceder a un fichero que no existe en la ruta indicada.
- **ImportError** : Ocurre cuando falla la importación de un módulo.

Manejo de excepciones con try, except, else y finally

Después de conocer los errores y las excepciones, aprenderemos a manejarlos utilizando los bloques try, except, else, y finally.

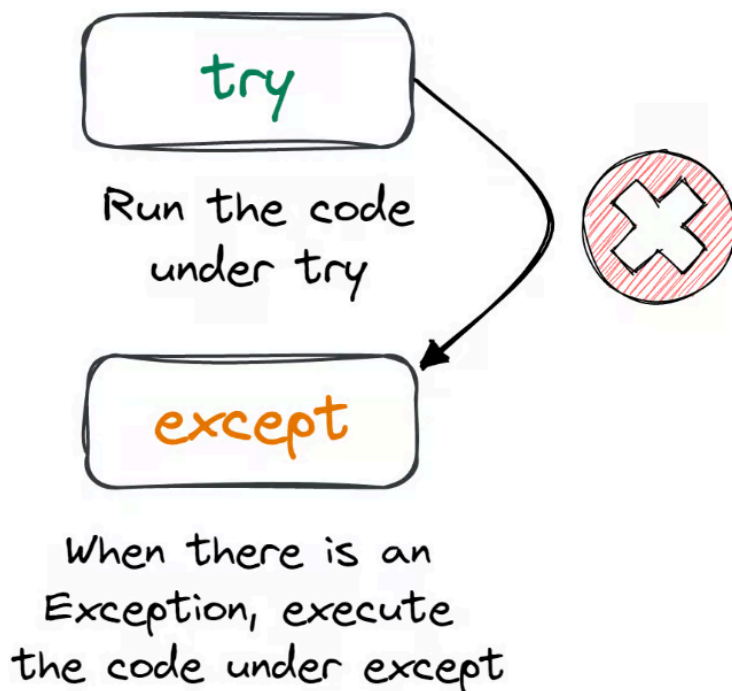
Entonces, ¿qué queremos decir con manejarlos? En circunstancias normales, estos errores detendrán la ejecución del código y mostrarán el mensaje de error. Para crear sistemas estables, tenemos que anticiparnos a estos errores e idear soluciones alternativas o mensajes de advertencia.

La sentencia try y except

La forma más sencilla de manejar excepciones en Python es utilizando el bloque try y except.

1. Ejecuta el código bajo la declaración try.
2. Cuando se produzca una excepción, ejecuta el código bajo la sentencia except.

En lugar de detenernos ante un error o una excepción, nuestro código avanzará hacia soluciones alternativas.



Ejemplo sencillo

En el primer ejemplo, intentaremos imprimir la variable indefinida x. En circunstancias normales, debería lanzar el error y detener la ejecución, pero con el bloque try y except, podemos cambiar el comportamiento del flujo.

- El programa ejecutará el código bajo la sentencia try.
- Como sabemos, ese x no está definido, por lo que ejecutará la sentencia except e imprimirá la advertencia.

Python

```
try:
    print(x)
except:
    print("An exception has occurred. X is unkwn!")
```

Ejemplo de sentencia except múltiple

En el segundo ejemplo, utilizaremos varias sentencias except para gestionar varios tipos de excepciones.

- Si se produce una excepción `ZeroDivisionError`, el programa imprimirá "No se puede dividir un valor por cero".
- El resto de las excepciones imprimirán "Algo más salió mal".

Nos permite escribir código flexible que puede manejar varias excepciones a la vez sin romperse.

Python

```
try:
    print(1/0)
except ZeroDivisionError:
    print("You cannot divide a value with zero")
except:
    print("Something else went wrong")
```

Output:

You cannot divide a value with zero

Cargar el archivo de ejemplo

Veamos ahora un ejemplo más práctico.

En el código siguiente, estamos leyendo el archivo CSV, y cuando se produzca la excepción `FileNotFoundError`, el código imprimirá el error y un mensaje adicional sobre el archivo `data.csv`.

Sí, podemos imprimir mensajes de error predeterminados sin interrumpir la ejecución.

```
Python

try:

    with open('data.csv') as file:

        read_data = file.read()

except FileNotFoundError as fnf_error:

    print(fnf_error)

    print("Explanation: We cannot load the 'data.csv' file")
```

Output:

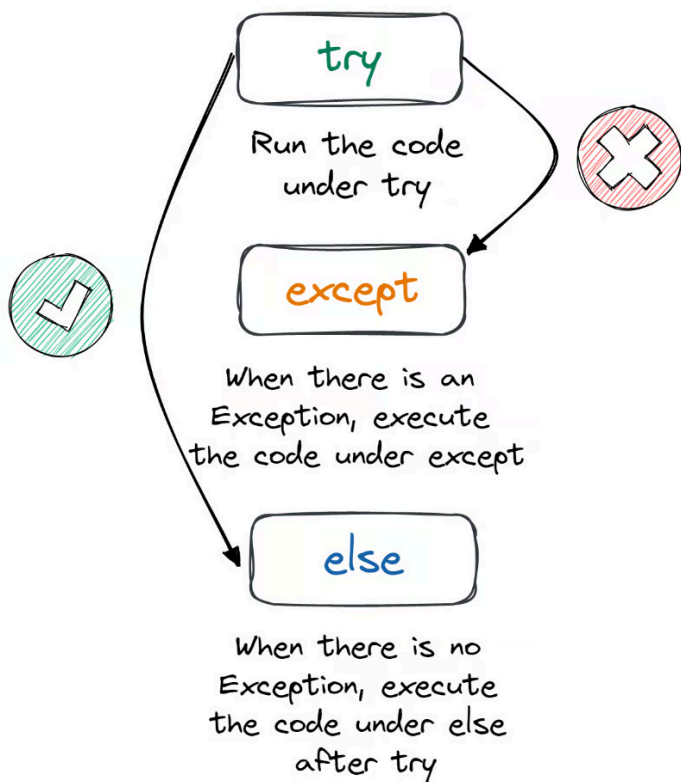
```
[Errno 2] No such file or directory: 'data.csv'
Explanation: We cannot load the 'data.csv' file
```

La cláusula try con else

Hemos aprendido sobre try y except, y ahora aprenderemos sobre la declaración else.

Cuando la sentencia try no lanza una excepción, el código entra en el bloque else. Es el remedio o una opción alternativa cuando esperas que una parte de tu script produzca una excepción. Generalmente se utiliza en una breve sección de configuración o verificación en la que no quieres que se oculten ciertos errores.

Nota: En el bloque try-except, puedes utilizar else después de todas las sentencias except.



Ejemplo sencillo

Estamos añadiendo la declaración else al ejemplo ZeroDivisionError. Como podemos ver, cuando no hay excepciones, se ejecuta la función de impresión bajo la sentencia else, mostrando el resultado.

Python

```
try:
    result = 1/3
except ZeroDivisionError as err:
    print(err)
else:
    print(f"Your answer is {result}")
```


Output:

Your answer is 0.3333333333333333

Ejemplo de IndexError con else

Vamos a aprender más creando una función sencilla y probándola en varios escenarios.

La función `find_nth_value()` tiene como argumentos `x` (lista) y `n` (número de índice). Hemos creado un bloque `try`, `except`, y `else` para gestionar la excepción `IndexError`.

```
Python

x = [5,8,9,13]

def find_nth_value(x,n):
    try:
        result = x[n]
    except IndexError as err:
        print(err)
    else:
        print("Your answer is ", result)

find_nth_value(x,6)
find_nth_value(x,2)
```

Output:

```
list index out of range
Your answer is 9
```

- En `n=6`, se lanzó la excepción `IndexError`, y llegamos a ver el mensaje de error por defecto "índice de lista fuera de rango".
- En `n=2`, no se produjo ninguna excepción, y la función imprimió el resultado que está debajo de la declaración `else`.

La palabra clave finally en Python

La palabra clave finally del bloque try-except se ejecuta siempre, independientemente de si hay una excepción o no. En palabras sencillas, el bloque de código finally se ejecuta después de que el bloque try, except y else hayan finalizado. Es muy útil para limpiar los recursos y cerrar el objeto, sobre todo para cerrar los archivos.



La función divide se crea para gestionar las excepciones de `ZeroDivisionError` y mostrar el resultado cuando no hay excepciones. Sea cual sea el resultado, siempre se ejecutará finally para imprimir "Code by USS" en color verde.

Python

```
def divide(x,y):  
    try:  
        result = x/y  
    except ZeroDivisionError:  
        print("Please change 'y' argument to non-zero value")  
    except:  
        print("Something went wrong")  
    else:  
        print(f"Your answer is {result}")  
    finally:  
        print("\033[92m Code by USS\033[00m")
```

En la primera prueba, estamos dividiendo 1 entre 0, lo que debería lanzar la excepción `ZeroDivisionError` e imprimir el mensaje. Como podemos ver, tenemos una línea adicional después del mensaje de error.

Python

```
divide(1,0)
```

None

Output:

```
Please change 'y' argument to non-zero value  
[92m Code by USS[00m]
```

Cuando añadimos una entrada válida, muestra el resultado ejecutando else y finally bloqueando.

Python

```
divide(3,4)
```

None

Your answer is 0.75

[92m Code by USS[00m]

Type Hints en Python

¿Qué son los Type Hints?

Los type hints (anotaciones de tipo) son una característica de Python 3.5+ que te permite indicar el tipo de dato esperado para variables, parámetros y valores de retorno. No son obligatorios — Python sigue siendo dinámico — pero son fundamentales en código profesional y en Django moderno.

¿Por qué son importantes en Django?

Todo el código de Django 5 usa type hints. Los serializers de DRF, los modelos, las views — todos usan anotaciones. Si no los conoces, el código de Django te parecerá extraño.

Sintaxis Básica

Python

Sin type hints (version basica)

```
def calcular_tarifa(peso, tipo_servicio):  
    tarifas = {"ESTANDAR": 5.0, "EXPRESS": 15.0}  
    return tarifas[tipo_servicio] + (peso * 2.5)
```

Con type hints (version profesional)

```
def calcular_tarifa(peso: float, tipo_servicio: str) -> float:  
    """  
    Calcula la tarifa de una encomienda.  
    Args:  
        peso: Peso en kilogramos  
        tipo_servicio: "ESTANDAR", "EXPRESS" o "ECONOMICO"  
    Returns:
```

```

    Tarifa en soles (S/.)
    """

    tarifas = {"ESTANDAR": 5.0, "EXPRESS": 15.0, "ECONOMICO": 3.0}

    base = tarifas.get(tipo_servicio, 5.0)

    return base + (peso * 2.5)

# Llamada
tarifa = calcular_tarifa(2.5, "EXPRESS") # -> 22.25

```

Tipos Básicos

```

Python

from typing import Optional, List, Dict, Tuple

# Tipos primitivos
codigo: str = "ENC-001"
peso: float = 2.5
activo: bool = True
cantidad: int = 10

# Listas tipadas
estados: List[str] = ["PENDIENTE", "EN_TRANSITO", "ENTREGADO"]

# Diccionarios tipados
tarifa_por_tipo: Dict[str, float] = {

```

```

    "ESTANDAR": 5.0,

    "EXPRESS": 15.0,

    "ECONOMICO": 3.0,
}

# Optional: puede ser el tipo indicado o None
telefono: Optional[str] = None # Puede no tener telefono

# Python 3.10+ (sintaxis simplificada)
telefono: str | None = None # Equivalente a Optional[str]

```

Type hints en clases (como Django los usa)

```

Python

from typing import Optional, List, Dict, Tuple

# Tipos primitivos
codigo: str = "ENC-001"

peso: float = 2.5

activo: bool = True

cantidad: int = 10

# Listas tipadas
estados: List[str] = ["PENDIENTE", "EN_TRANSITO", "ENTREGADO"]

```

```
# Diccionarios tipados
```

```
tarifa_por_tipo: Dict[str, float] = {  
    "ESTANDAR": 5.0,  
    "EXPRESS": 15.0,  
    "ECONOMICO": 3.0,  
}
```

```
# Optional: puede ser el tipo indicado o None
```

```
telefono: Optional[str] = None # Puede no tener telefono
```

```
# Python 3.10+ (sintaxis simplificada)
```

```
telefono: str | None = None # Equivalente a Optional[str]
```


Statements y Control de Flujo

Condicionales — if / elif / else

Los condicionales controlan que bloque de código se ejecuta según una condición. En Django los usamos constantemente en views y services para aplicar lógica de negocio.

Python

Ejemplo aplicado: validar el estado de una encomienda

```
estado = "EN_TRANSITO"
```

```
if estado == "REGISTRADA":
```

```
    print("La encomienda acaba de registrarse.")
```

```
elif estado == "EN_TRANSITO":
```

```
    print("La encomienda va en camino.")
```

```
elif estado == "EN_AGENCIA_DESTINO":
```

```
    print("Ya llego a la ciudad de destino.")
```

```
elif estado == "ENTREGADA":
```

```
    print("Encomienda entregada exitosamente.")
```

```
else:
```

```
    print("Estado desconocido o cancelado.")
```

Operadores de comparación y logicos

Python

Operadores de comparacion

== igual a

```
# != diferente de

# > mayor que

# < menor que

# >= mayor o igual que

# <= menor o igual que


# Ejemplo: validar si una encomienda es valida para envio express

peso_kg = 2.5

tipo_servicio = "EXPRESS"

valor_declarado = 500


# Operadores logicos: and, or, not

if tipo_servicio == "EXPRESS" and peso_kg <= 30:

    print("Puede enviarse como Express.")


if peso_kg > 30 or valor_declarado > 5000:

    print("Requiere aprobacion especial.")


if not (estado == "CANCELADA"):

    print("Encomienda activa.")
```

Bucle for

El bucle for, itera sobre cualquier secuencia: listas, tuplas, diccionarios, rangos. Es el mas usado en Python.

Python

Iterar sobre una lista de encomiendas

```
encomiendas = [  
    {"codigo": "ENC-001", "peso": 2.5, "estado": "EN_TRANSITO"},  
    {"codigo": "ENC-002", "peso": 0.8, "estado": "ENTREGADA"},  
    {"codigo": "ENC-003", "peso": 15.0, "estado": "REGISTRADA"},  
]
```

```
for enc in encomiendas:
```

```
    print(f"Codigo: {enc['codigo']} | Estado: {enc['estado']}")
```

Output:

Codigo: ENC-001 | Estado: EN_TRANSITO

Codigo: ENC-002 | Estado: ENTREGADA

Codigo: ENC-003 | Estado: REGISTRADA

Python

for con range(): iterar un numero especifico de veces

```
for i in range(5):
```

```
    print(i) # 0, 1, 2, 3, 4
```

range(inicio, fin, paso)

```
for i in range(1, 10, 2):
```

```
    print(i) # 1, 3, 5, 7, 9
```

for con enumerate(): indice + valor

```
ciudades = ["Lima", "Arequipa", "Trujillo", "Cusco"]

for indice, ciudad in enumerate(ciudades, start=1):

    print(f"{indice}. {ciudad}")

# Output:

# 1. Lima

# 2. Arequipa ... etc


# for sobre diccionario

tarifas = {"ESTANDAR": 5.0, "EXPRESS": 15.0, "ECONOMICO": 3.0}

for tipo, precio in tarifas.items():

    print(f"Tipo {tipo}: S/. {precio:.2f}")
```

Bucle while

El bucle while se encarga de repetir la ejecución de un bloque de código siempre que una condición específica se evalúe como verdadera. En el desarrollo backend, esta estructura es fundamental para gestionar lógicas de reintentos, técnicas de polling o el procesamiento de datos por lotes.

Python

```
# Simular reintentos de entrega de una encomienda

intentos = 0

max_intentos = 3

entregada = False
```

```
while intentos < max_intentos and not entregada:

    intentos += 1

    print(f"Intento {intentos} de entrega...")

    # Simulacion: entrega exitosa en el 2do intento
    if intentos == 2:

        entregada = True

        print("Encomienda entregada exitosamente.")

if not entregada:

    print(f"No se pudo entregar despues de {max_intentos} intentos.")

# Output:
# Intento 1 de entrega...
# Intento 2 de entrega...
# Encomienda entregada exitosamente.
```

Programación Orientada a Objetos

Por qué POO es fundamental para Django

En Django, CADA modelo es una clase (hereda de `models.Model`). CADA vista es una clase (hereda de `View` o `ViewSet`). Cada serializer es una clase (heredada de `Serializer`).

¿Qué es una Clase?

Una clase es un molde o plantilla para crear objetos. Define qué datos tiene (atributos) y que puede hacer (métodos). Un objeto es una instancia de esa clase.

Python

```
# Analogia: La clase "Encomienda" es el molde
# Cada encomienda especifica (ENC-001, ENC-002...) es un objeto

class Encomienda:
    """Representa una encomienda en el sistema."""

    # __init__: se ejecuta al crear un objeto (constructor)
    def __init__(self, codigo: str, peso: float, remitente: str):
        # self.atributo = valor -> define los datos del objeto
        self.codigo = codigo
        self.peso = peso
        self.remitente = remitente
        self.estado = "REGISTRADA" # valor por defecto

    # Metodo: accion que puede realizar el objeto
    def cambiar_estado(self, nuevo_estado: str) -> None:
```

```

        self.estado = nuevo_estado

        print(f"Encomienda {self.codigo} -> {nuevo_estado}")

# __str__: representacion en texto del objeto
def __str__(self) -> str:
    return f"[{self.codigo}] {self.remitente} | Estado: {self.estado}"

# Crear objetos (instanciar la clase)
enc1 = Encomienda("ENC-001", 2.5, "Juan Perez")
enc2 = Encomienda("ENC-002", 0.8, "Maria Garcia")

# Acceder a atributos
print(enc1.codigo)    # ENC-001
print(enc2.peso)      # 0.8

# Llamar metodos
enc1.cambiar_estado("EN_TRANSITO")

# Usar __str__
print(enc1)  # [ENC-001] Juan Perez | Estado: EN_TRANSITO

```

Atributos de Clase vs. Atributos de Instancia

Imagina que tienes un formulario impreso para registrar encomiendas. El formulario ya viene con información impresa (igual para todos): los estados posibles, las tarifas base, el nombre de la empresa. Ese es el atributo de clase.

Cuando llenas un formulario específico con los datos de una encomienda (el código, el peso, el remitente), esa información es única de ese papel. Eso es el atributo de instancia

Python

```
class Encomienda:

    # — ATRIBUTOS DE CLASE —
    # Se definen FUERA del __init__, directamente en el cuerpo de la clase.
    # Son COMPARTIDOS por todos los objetos.
    # Cambian si modificas la clase, no el objeto individual.

    TARIFAS_BASE = {
        "ESTANDAR": 5.0,
        "EXPRESS": 15.0,
        "ECONOMICO": 3.0,
    }

    ESTADOS_VALIDOS = ["REGISTRADA", "EN_TRANSITO", "ENTREGADA",
                       "CANCELADA"]

    # — ATRIBUTOS DE INSTANCIA —
    # Se definen DENTRO del __init__ usando self.
    # Son ÚNICOS para cada objeto.
    # Cada encomienda tiene su propio código, peso, estado.
```



```

def __init__(self, codigo: str, peso: float, tipo: str):
    self.codigo = codigo    # único de esta encomienda
    self.peso    = peso      # único de esta encomienda
    self.tipo    = tipo      # único de esta encomienda

    self.estado = "REGISTRADA" # empieza igual, pero puede cambiar
    independientemente

```

La diferencia clave: qué pasa al crear dos objetos

Python

```

enc1 = Encomienda("ENC-001", 2.5, "EXPRESS")
enc2 = Encomienda("ENC-002", 8.0, "ESTANDAR")

# Atributos de INSTANCIA - cada uno tiene el suyo propio
print(enc1.codigo)    # ENC-001
print(enc2.codigo)    # ENC-002    ← diferente

print(enc1.peso)      # 2.5
print(enc2.peso)      # 8.0        ← diferente

# Atributos de CLASE - ambos comparten el mismo
print(enc1.TARIFAS_BASE) # {"ESTANDAR": 5.0, "EXPRESS": 15.0, ...}
print(enc2.TARIFAS_BASE) # {"ESTANDAR": 5.0, "EXPRESS": 15.0, ...} ←
IGUAL

```

Lo que hace especial a los atributos de clase

Python

```
# Puedes acceder sin crear ningún objeto

print(Encomienda.ESTADOS_VALIDOS)  # ["REGISTRADA", "EN_TRANSITO", ...]

print(Encomienda.TARIFAS_BASE)      # {"ESTANDAR": 5.0, ...}

# Si cambias el atributo de clase, TODOS los objetos lo ven

Encomienda.TARIFAS_BASE["EXPRESS"] = 20.0  # subió la tarifa express

enc1 = Encomienda("ENC-001", 2.5, "EXPRESS")
enc2 = Encomienda("ENC-002", 1.0, "EXPRESS")

print(enc1.TARIFAS_BASE["EXPRESS"])  # 20.0 ← ambos ven el cambio
print(enc2.TARIFAS_BASE["EXPRESS"])  # 20.0
```

Properties — Atributos Calculados

El decorador `@property` permite definir un método que se accede como si fuera un atributo. Django lo usa para campos calculados.

Python

```
class Encomienda:
    def __init__(self, codigo, peso, largo=None, ancho=None, alto=None):
        self.codigo = codigo
        self.peso = peso
        self.largo = largo
        self.ancho = ancho
```

```

self.alto = alto

@property
def volumen_cm3(self):
    """Calcula el volumen si estan todas las dimensiones."""
    if all([self.largo, self.ancho, self.alto]):
        return self.largo * self.ancho * self.alto
    return None

@property
def es_paquete_grande(self) -> bool:
    return self.peso > 20.0

@property
def descripcion_corta(self) -> str:
    tipo = "Grande" if self.es_paquete_grande else "Pequeno"
    return f"{self.codigo} ({tipo}, {self.peso}kg)"

# Uso: se accede como atributo, no como metodo (sin parentesis)
enc = Encomienda("ENC-001", 25.0, largo=30, ancho=20, alto=15)
print(enc.volumen_cm3)      # 9000
print(enc.es_paquete_grande) # True
print(enc.descripcion_corta) # ENC-001 (Grande, 25.0kg)

```

Herencia — La base de Django

La herencia permite que una clase herede atributos y metodos de otra. En Django, tus modelos heredan de `models.Model`, tus vistas de `APIView`, etc. Debes entender herencia para entender Django

Python

```

# Clase base: comportamiento comun para todas las entidades
class EntidadBase:
    """Clase abstracta con campos de auditoria."""

    def __init__(self):
        import datetime
        self.creado_en = datetime.datetime.now()
        self.actualizado_en = datetime.datetime.now()
        self.activo = True

    def desactivar(self) -> None:
        self.activo = False
        import datetime
        self.actualizado_en = datetime.datetime.now()

    def __repr__(self) -> str:
        return f"{self.__class__.__name__}(activo={self.activo})"

# Clase hija: hereda de EntidadBase
class Agencia(EntidadBase):
    """Agencia de encomiendas en una ciudad."""

    def __init__(self, nombre: str, ciudad: str, codigo: str):
        super().__init__() # Llama al __init__ del padre
        self.nombre = nombre
        self.ciudad = ciudad
        self.codigo = codigo
        self.es_hub = False

    def __str__(self) -> str:
        return f"{self.nombre} - {self.ciudad}"

class AgenciaHub(Agencia):
    """Agencia hub: centro de distribucion regional."""

    def __init__(self, nombre: str, ciudad: str, codigo: str, region: str):
        super().__init__(nombre, ciudad, codigo)
        self.es_hub = True # Sobreescribe el valor del padre

```

```

self.region = region
self.agencias_dependientes = []

def agregar_agencia(self, agencia: Agencia) -> None:
    self.agencias_dependientes.append(agencia)

def __str__(self) -> str:
    return f"HUB {self.nombre} ({self.region}) - {len(self.agencias_dependientes)} agencias"

# Uso
ag1 = Agencia("Agencia Lima Centro", "Lima", "LIM-001")
hub = AgenciaHub("Hub Lima Norte", "Lima", "LIM-HUB", "Lima Metropolitana")

hub.agregar_agencia(ag1)

print(ag1)    # Agencia Lima Centro - Lima
print(hub)    # HUB Hub Lima Norte (Lima Metropolitana) - 1 agencias

# Herencia: AgenciaHub tiene metodos de EntidadBase
hub.desactivar()    # Heredado de EntidadBase
print(hub.activo)    # False

```

Metodos especiales (__dunder__ methods)

Los dunder methods (double underscore = "dunder") son métodos especiales que Python llama automáticamente en situaciones específicas. Tú no los invocas directamente — Python los invoca por ti cuando usas operadores, funciones built-in o ciertas sintaxis del lenguaje.

La idea central

Cuando escribes `print(enc)`, Python internamente llama `enc.__str__()`. Cuando escribes `enc1 == enc2`, Python llama `enc1.__eq__(enc2)`. Cuando escribes `len(enc)`, Python llama `enc.__len__()`.

Son los "ganchos" que conectan tus clases con el comportamiento nativo de Python.

Los más importantes para Django

`__init__` — El constructor

Python

```
class Encomienda:

    def __init__(self, codigo: str, peso: float):

        self.codigo = codigo

        self.peso = peso

# Python llama __init__ automáticamente al hacer esto:

enc = Encomienda("ENC-001", 2.5)

# Internamente: Encomienda.__init__(enc, "ENC-001", 2.5)
```

`__str__` — Lo que ve el usuario

Se llama cuando usas `print()` o `str()`. Es la representación legible para humanos.

Python

```
class Encomienda:

    def __init__(self, codigo, peso, estado):

        self.codigo = codigo

        self.peso = peso

        self.estado = estado
```

```

def __str__(self) -> str:
    return f"Encomienda {self.codigo} | {self.peso}kg | {self.estado}"

enc = Encomienda("ENC-001", 2.5, "EN_TRANSITO")

print(enc)          # Encomienda ENC-001 | 2.5kg | EN_TRANSITO
print(str(enc))     # Encomienda ENC-001 | 2.5kg | EN_TRANSITO
f"Info: {enc}"      # "Info: Encomienda ENC-001 | 2.5kg | EN_TRANSITO"

```

En el ecosistema de Django, la implementación del método `__str__` resulta fundamental para cada modelo, puesto que constituye la representación legible que el admin panel empleará al visualizar las instancias.

`__repr__` — La representación para el desarrollador

Este método se invoca al inspeccionar el objeto en el shell de Python o en archivos de logs, y actúa como respaldo automático si `__str__` no está presente. Su objetivo es ofrecer una descripción técnica y precisa que facilite la depuración, buscando idealmente ser una salida reproducible para el programador.

Python

```

class Encomienda:

    def __init__(self, codigo, peso):
        self.codigo = codigo
        self.peso = peso

    def __str__(self) -> str:
        return f"Encomienda {self.codigo}"    # Para el usuario

```

```

def __repr__(self) -> str:
    return f"Encomienda(codigo={self.codigo!r}, peso={self.peso})" #
Para el dev

enc = Encomienda("ENC-001", 2.5)

print(enc)      # Encomienda ENC-001          <- usa __str__
repr(enc)       # Encomienda(codigo='ENC-001', peso=2.5) <- usa __repr__

# En el shell de Python, al escribir solo "enc" sin print:
# >>> enc
# Encomienda(codigo='ENC-001', peso=2.5) <- usa __repr__

```

`__eq__` — El operador `==`

Sin `__eq__`, Python compara si dos variables apuntan al **mismo objeto en memoria**, no si tienen los mismos datos.

Python

```

class Encomienda:
    def __init__(self, codigo, peso):
        self.codigo = codigo
        self.peso = peso

```



```

enc1 = Encomienda("ENC-001", 2.5)
enc2 = Encomienda("ENC-001", 2.5)

# Sin __eq__: compara posición en memoria
print(enc1 == enc2)    # False ← son dos objetos distintos aunque tengan los
                        # mismos datos!

print(enc1 is enc2)    # False

# Con __eq__: compara por el campo que tú definas
class Encomienda:
    def __init__(self, codigo, peso):
        self.codigo = codigo
        self.peso = peso

    def __eq__(self, other) -> bool:
        if not isinstance(other, Encomienda):
            return False
        return self.codigo == other.codigo # Igualdad por código

enc1 = Encomienda("ENC-001", 2.5)
enc2 = Encomienda("ENC-001", 8.0)    # mismo código, distinto peso
enc3 = Encomienda("ENC-002", 2.5)    # distinto código

print(enc1 == enc2)    # True ← mismo código
print(enc1 == enc3)    # False ← diferente código

```

Python

```
class Encomienda:
```

```
    def __init__(self, codigo: str, peso: float):
```

```
        self.codigo = codigo
```

```
        self.peso = peso
```

```
    # Representacion para print()
```

```
    def __str__(self) -> str:
```

```
        return f"Encomienda {self.codigo}"
```

```
    # Representacion para debug (Python shell)
```

```
    def __repr__(self) -> str:
```

```
        return f"Encomienda(codigo={self.codigo!r}, peso={self.peso})"
```

```
    # Comparacion de igualdad (==)
```

```
    def __eq__(self, other) -> bool:
```

```
        if not isinstance(other, Encomienda):
```

```
            return False
```

```
        return self.codigo == other.codigo
```

```
    # len(encomienda) -> peso en gramos
```

```
    def __len__(self) -> int:
```

```
        return int(self.peso * 1000)
```

```
enc1 = Encomienda("ENC-001", 2.5)
```

```
enc2 = Encomienda("ENC-001", 3.0)
enc3 = Encomienda("ENC-002", 1.0)

print(str(enc1))      # Encomienda ENC-001
print(repr(enc1))     # Encomienda(codigo="ENC-001", peso=2.5)
print(enc1 == enc2)   # True (mismo codigo)
print(enc1 == enc3)   # False (diferente codigo)
print(len(enc1))      # 2500 (gramos)
```

Instalación de Python

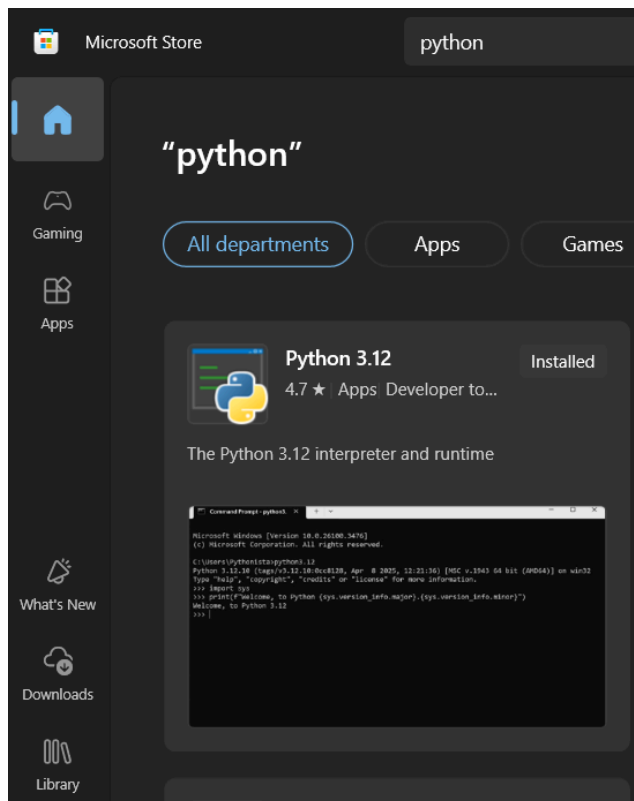
Hay varias formas de instalar Python en una máquina Windows. A continuación encontrarás las opciones que explicaremos en este tutorial:

- Instalar Python directamente desde Microsoft Store: esta opción rápida y sencilla te pondrá a trabajar con Python en un abrir y cerrar de ojos. Es especialmente útil para los principiantes que quieran utilizar Python en su equipo con fines de aprendizaje.
- Instalar Python directamente desde el sitio web de Python: este método te da más control sobre el proceso de instalación y te permite personalizarla.

Instalando Python desde Microsoft Store

1. Abre la aplicación Microsoft Store en tu equipo Windows. Puedes hacerlo haciendo clic en el menú Inicio y buscando "Microsoft Store".
2. En la aplicación Microsoft Store, busca "Python". Deberías ver diferentes aplicaciones "Python X.X" en los resultados de la búsqueda. Con X.X nos referimos a las diferentes versiones de Python disponibles en Microsoft Store. Para efectos de este curso, usaremos la 3.12
3. Pulsa en la aplicación para abrir la página de la aplicación.
4. Pulsa el botón "Get" para iniciar el proceso de instalación.

5. La Microsoft Store descargará e instalará Python en tu equipo. Esto puede tardar unos minutos, dependiendo de la velocidad de tu conexión a Internet.
6. Una vez finalizada la instalación, sigue las instrucciones de la sección "Comprobar si Python ya está instalado en tu equipo Windows" para comprobar que Python se ha instalado correctamente.
7. Esta instalación de Python también incluye el IDLE Shell, que es un sencillo IDE para ejecutar comandos de Python. Para acceder a ella, busca IDLE Shell en la sección "Todas las aplicaciones" del menú Inicio.



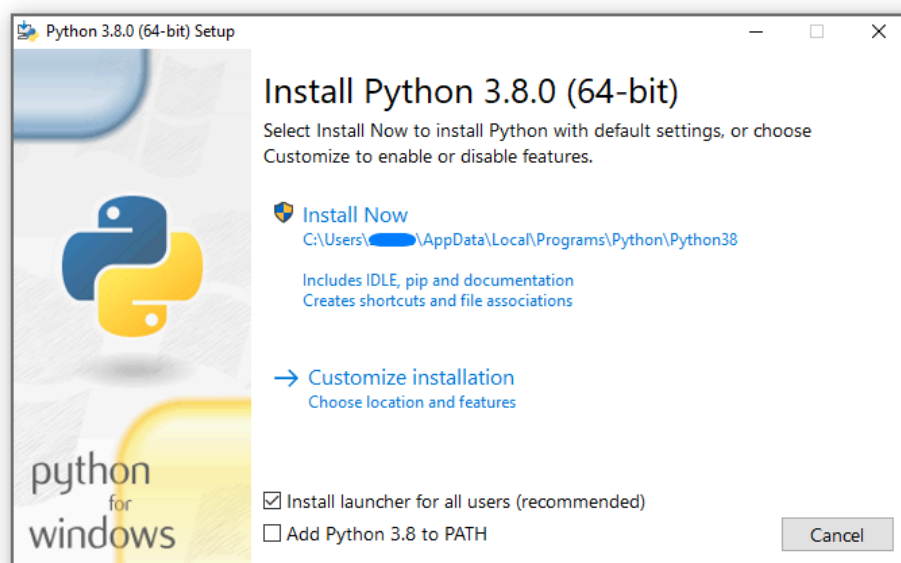
Como ya hemos dicho, la instalación de Python en Microsoft Store es útil sobre todo para fines educativos. Como se indica en la propia documentación de Python, "El paquete Microsoft Store es un intérprete de Python fácil de instalar que está pensado principalmente para uso interactivo, por ejemplo, para estudiantes".

Instalando Python desde el sitio oficial de Python.org

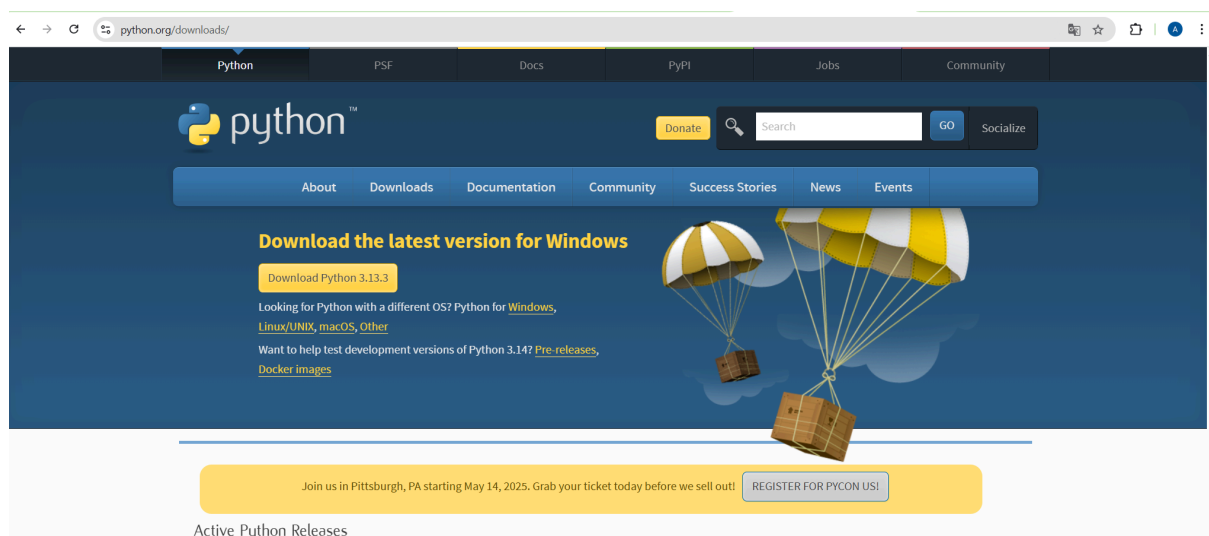
Para instalar Python en tu equipo Windows utilizando el instalador completo de Python del sitio web de Python, realiza los siguientes pasos:

1. Ve a la página de descarga de Python. <https://www.python.org/downloads/>

2. En la sección "Python Releases for Windows", haz clic en el enlace correspondiente a la última versión de Python (por ejemplo, en el momento de escribir esto, la última versión de Python es Python 3.11.1).
3. En la página de descarga de Python, haz clic en la versión "Windows installer (64-bit)" de Python.
4. Una vez finalizada la descarga, ejecuta el archivo instalador para iniciar el proceso de instalación.
5. Una vez que se abra el instalador, verás una opción para "Add Python 3.x to PATH". Esto sólo se recomienda si sólo vas a instalar una versión de Python (en lugar de varias versiones) y quieres utilizar Python desde el terminal (en lugar de desde un IDE).



6. Pulsa el botón "Install Now" para iniciar la instalación. El proceso de instalación durará unos minutos. Ten en cuenta que antes de hacer clic en "Install Now", es posible que se te pida que elijas las funciones que deseas como parte de la instalación. A menos que tengas preferencias específicas de personalización, se recomienda proceder con la instalación por defecto.
7. Cuando finalice el proceso de instalación, se te dará la opción de 'Disable path length limit'. Para el uso general de Python no lo necesitas, pero para el desarrollo de Python, es recomendable, ya que puede evitar problemas con las largas rutas de archivos autogeneradas.
8. Pulsa el botón "Close" para cerrar el instalador.
9. Una vez finalizada la instalación, sigue las instrucciones de la sección "Comprobar si Python ya está instalado en tu equipo Windows" para comprobar que Python se ha instalado correctamente.



Cómo instalar paquetes Python

Python es modular, con un gran ecosistema de paquetes que proporcionan funcionalidad para tareas específicas de la ciencia de datos. Por ejemplo, el paquete pandas proporciona funcionalidad para la manipulación de datos, scikit-learn proporciona funcionalidad de machine learning, y PyTorch proporciona funcionalidad de aprendizaje profundo.

Existen dos herramientas de gestión de paquetes para instalar paquetes de Python: pip3 y conda. Estas herramientas te permiten instalar y actualizar paquetes de Python.

Instalar paquetes con pip3

Utiliza pip3 si instalaste Python desde el sitio web de Python o desde Microsoft Store.

Para instalar paquetes con pip3, sigue estos pasos:

1. Antes de intentar instalar los paquetes de Python, asegúrate de que Python está instalado en tu equipo. Para instalar Python, sigue las instrucciones de una de las secciones anteriores (como descargar e instalar desde el sitio web utilizando Microsoft Store).
2. Para instalar un paquete utilizando pip3, abre un Terminal en macOS o un Símbolo del sistema en Windows y escribe el siguiente comando:

Python

```
pip3 install {package_name}
```

3. Si el paquete tiene dependencias (es decir, necesita otros paquetes para funcionar), pip3 también las instalará automáticamente.
4. Una vez completada la instalación, puedes importar el paquete en tu código Python. Por ejemplo, si has instalado el paquete numpy, podrías importarlo y utilizarlo así:

Python

```
import numpy as np  
  
arr = np.array(["I", "love", "Python", "package", "management"])
```

5. Si quieres actualizar un paquete a la última versión, puedes utilizar el comando pip3 install --upgrade.

Python

```
pip3 install --upgrade {package_name}
```

6. Para listar todos los paquetes instalados, utiliza el comando pip3 freeze.

Python

```
pip3 freeze
```

En algunas ocasiones, cuando desarrollamos necesitamos listar y generar todos los paquetes que hemos utilizado en el proyecto para poder moverlo a otro ambiente por ejemplo, para ello usaremos el siguiente comando:

Python

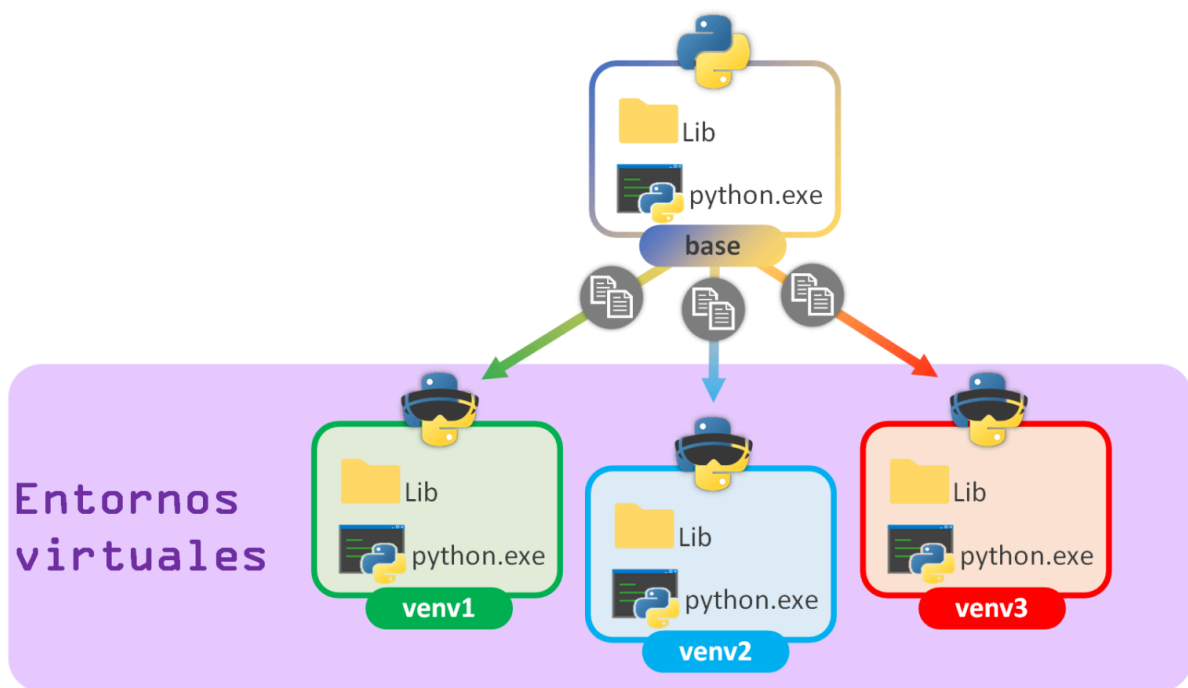
```
python -m pip freeze > requirements.txt
```

Entornos Virtuales en Python

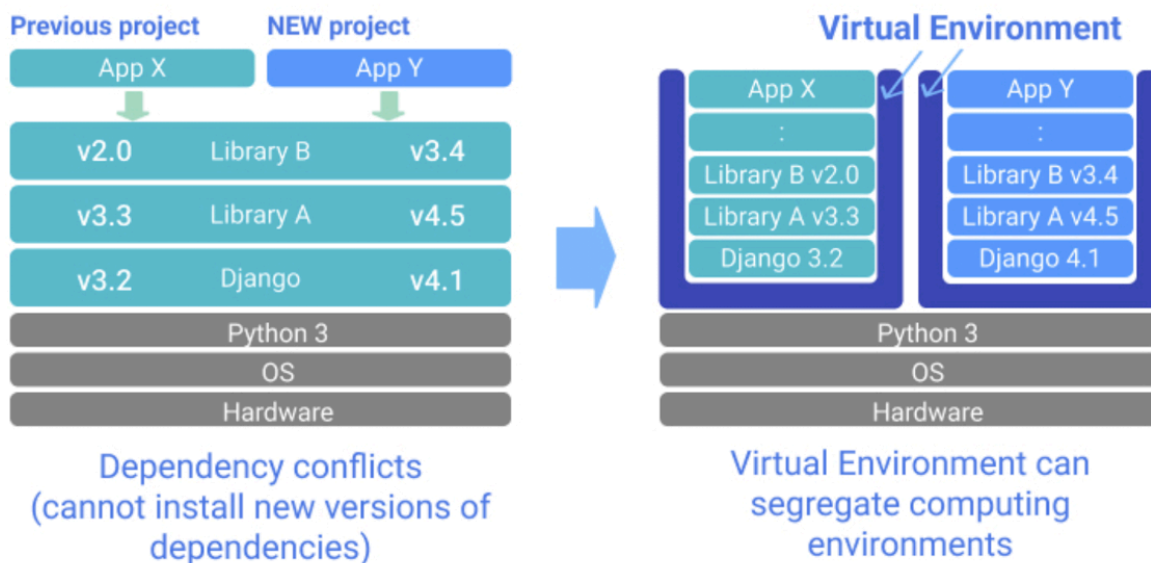
Al desarrollar o ejecutar una aplicación, ésta utiliza un conjunto de programas de software interconectados (denominados "dependencias").

Por ejemplo, su ordenador utiliza un sistema operativo específico (p. ej., Mac OS X Yosemite). En dicho sistema operativo, se instalan versiones específicas del lenguaje de programación (p. ej., Python 3.8), un framework web (p. ej., Django 3.2) o diversas bibliotecas. Al desarrollar una aplicación Django, es importante prestar atención a las dependencias.

Su aplicación puede ejecutarse correctamente con las dependencias configuradas; sin embargo, es posible que no funcione correctamente en otro computador, ya que las versiones de las bibliotecas, Python o el propio Django pueden variar entre computador. El entorno que depende de cada computador se denomina entorno informático.



Un entorno virtual es un concepto o herramienta que se utiliza para crear un entorno específico para una aplicación, separándolo de otros recursos. Existen varias maneras de crear un entorno virtual. Una opción es usar software de máquina virtual. El enfoque más popular recientemente es usar un contenedor Docker. Sin embargo, estos enfoques requieren tiempo para configurarse. Python cuenta con una función para crear un entorno virtual mediante la ejecución de un comando simple.



Creando un entorno virtual

1. Vamos a crear una carpeta denominada sesion02, ejecutamos los siguientes comandos en la línea de comandos:

None

```
d:>md {apellido_estudiante}
d:>cd {apellido_estudiante}
d:>md sesion02
d:>cd sesion02
```

2. Dentro de la carpeta vamos a crear nuestro entorno virtual ejecutando la siguiente instrucción

Python

```
python -m venv {nombre_entorno}
```

Por buenas prácticas los entornos debemos nombrarlos como env. Si bien es cierto podemos definir el nombre que deseemos, te recomiendo siempre seguir una convención .

3. Una vez el entorno haya sido creado, lo siguiente será activarlo. La activación depende completamente del sistema operativo. Aquí los comandos que necesitas.

Python

```
##para windows

{nombre_entorno}\\Scripts\\activate

##para unix/mac

source {nombre_entorno}/bin/activate
```

4. Una vez activado el entorno, en nuestra línea de comando nos debe aparecer algo así:

None

```
(env)d:>{carpeta_proyecto}\
```

5. Para desactivarlo ejecutamos el siguiente comando

None

```
d:>deactivate
```

Laboratorio: Setup completo del proyecto

1. Crear el directorio y entorno virtual

None

```
mkdir sistema_encomiendas && cd sistema_encomiendas  
python3 -m venv venv  
source venv/bin/activate      # Linux/Mac
```

1. Instalar dependencias base

None

```
pip install django==5.0.6 djangorestframework==3.15.2 django-environ==0.11.2  
pip freeze > requirements.txt
```